Documentación
Proyecto Intermodular

Módulo: PROYECTO INTERMODULAR

Fecha: 15/04/2026

Elaborado: Alonso De La Nuez, Kevin
Burgos Mac Colm, Iván
Leonardo Romero, Marina
Ortiz Gutiérrez, Simón

Tabla de Contenidos

1 OBJETIVOS Y ALCANCE 5

1.1 Objetivo 5

1.2 Objetivos Secundarios 5

1.3 Alcance 6

2 GESTIÓN DE PROYECTO 9

2.1 Introducción 9

2.2 Metodología de Trabajo 10

2.3 Fases del Proyecto 11

2.4 Planificación Temporal 12

2.5 Actividades y tareas 13

2.6 Recursos Necesarios 14

2.7 Permisos y Autorizaciones 15

2.8 Procedimiento de Ejecución de Tareas 15

2.9 Seguimiento y Control 16

2.10 Indicadores de Control 18

2.11 Participación en la evaluación 18

3 GESTIÓN DE LA CONFIGURACIÓN 19

3.1 Introducción 19

3.2 Aplicación Android 20

3.3 Servidor 21

3.4 Componente de Acceso a Datos 22

3.5 Base de Datos 23

3.6 Documentación del Proyecto 24

3.7 Nomenclatura de Versiones 24

3.8 Sistema de Almacenamiento 25

3.9 Metadatos de cada versión de EC 26

3.10 Registro de la Incidencia 27

3.11 Evaluación de la Incidencia 27

3.12 Solucionar la Incidencia 28

4 ANÁLISIS Y GESTIÓN DEL RIESGO 29

4.1 Introducción 29

4.2 Identificación de Riesgos 30

4.3 Evaluación de Riesgos 32

4.4 Gestión de Riesgos 33

4.5 Seguimiento del Riesgo 34

5 ASEGURAMIENTO DE LA CALIDAD 35

5.1 Introducción 35

5.2 Objetivos del Aseguramiento de la Calidad 35

5.3 Productos que se deben controlar 36

5.4 Control de Calidad por Producto 36

5.5 Procedimiento de Participación de los Usuarios 43

5.6 Gestión de Incidencias 44

5.7 Criterios de Aceptación Final 45

6 ANÁLISIS 46

6.1 Especificación de Requisitos 46

6.2 Estudio de Viabilidad Técnica 56

6.3 Diagrama de Casos de Uso 62

6.4 Modelo Entidad Relación 66

7 DISEÑO 68

7.1 Arquitectura del Sistema 68

7.2 Diagrama de Clases 71

7.3 Modelo Relacional 74

8 MATRICES DE TRAZABILIDAD 77

8.1 Requisitos: Casos de uso 79

8.2 Requisitos: Diseño (clases) 80

8.3 Requisitos: Pruebas 81

1 OBJETIVOS Y ALCANCE

1.1 Objetivo

El juego se desarrolla en Android Studio y está destinado a ejecutarse como una aplicación móvil para dispositivos Android.

Se trata de un juego RPG por niveles en el que el jugador desciende por una mazmorra con cierta cantidad de pisos, enfrentando enemigos aleatorios en cada piso. Cada combate otorga dinero, que permite mejorar al personaje y comprar objetos en la tienda para afrontar desafíos cada vez más complejos. Si el jugador no puede superar un piso, puede descender para repetir peleas y recolectar recursos o mejorar su estrategia.

El jugador puede gastar su dinero para comprar armas, armaduras, objetos y habilidades, además de mejorar su daño base, defensa base y vida máxima.

1.2 Objetivos Secundarios

- Diseñar un sistema de progresión escalable hasta el piso número 20.
- Implementar un sistema de combate por turnos con toma de decisiones estratégicas.
- Desarrollar un sistema de monedas que permita mejorar al personaje adquiriendo objetos, armas, armaduras y habilidades.
- Crear una tienda interactiva donde el jugador pueda adquirir equipamiento, objetos y habilidades.
- Gestionar un sistema de inventario funcional con objetos consumibles.
- Implementar enemigos con dificultad progresiva y habilidades únicas.
- Permitir la personalización del personaje mediante estadísticas y equipamiento.
- Diseñar un sistema de estadísticas para administradores que permita analizar el comportamiento de los jugadores.
- Garantizar una experiencia única mediante una jugabilidad entretenida y dinámica.

1.3 Alcance

Descripción funcional del sistema de juego

El sistema estará diseñado para gestionar una experiencia de juego por pisos, incorporando elementos de rol, progresión estratégica.

Las funcionalidades clave incluyen:

Gestión de usuarios

- El sistema distinguirá entre dos tipos de usuarios:
    - **Jugadores**: cada uno controlará un único personaje, con acceso a su inventario.
    - **Administradores**: Visualizar estadísticas de los diferentes jugadores.

Gestión de personajes

- Cada personaje contará con atributos personalizables y evolutivos:
    - Vida máxima, daño base y defensa base con opción a mejora.
    - Equipamiento: Armas y armadura
- Habilidades activas, desbloqueables mediante compras en la tienda.
- Inventario de objetos: consumibles comprados en la tienda con diferente cantidad de usos.

Sistema de tienda y mejoras

- Los jugadores podrán acceder a una tienda donde podrán:
    - Comprar armas, armaduras, objetos y habilidades.
    - Realizar mejoras de ataque base, defensa base y vida máxima mediante dinero obtenido en combate.

Gestión de batallas por turnos

- Las batallas se desarrollarán en formato por turnos, efectos de estado y uso estratégico de habilidades.
- Cada encuentro se resolverá mediante cálculos de daño y defensa, considerando el equipamiento del personaje también teniendo en cuenta ventajas y desventajas (Aplicadas por objetos o habilidades).

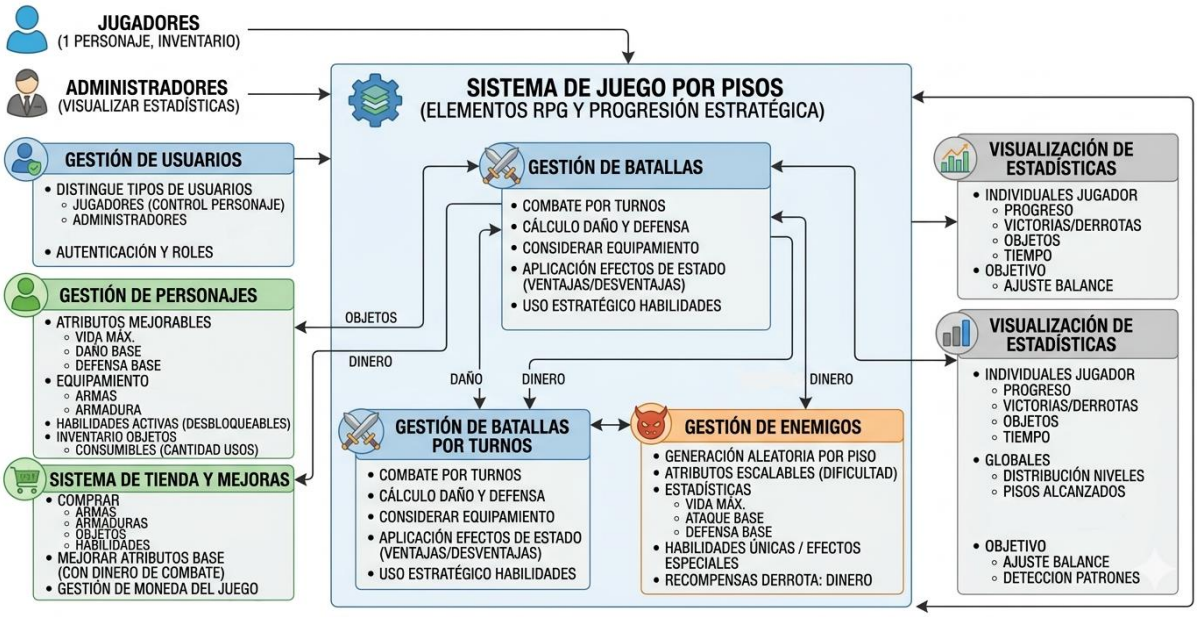
Gestión de enemigos

- Cada piso contará con enemigos generados aleatoriamente, con atributos escalables según la dificultad del nivel.
- Los enemigos tendrán:
    - Estadísticas propias vida máxima, ataque base y defensa base.
    - Habilidades únicas o efectos especiales que obligan al jugador a adaptar su estrategia.
- Al ser derrotados, los enemigos otorgarán:
    - Dinero para usar en tienda.

Visualización de estadísticas

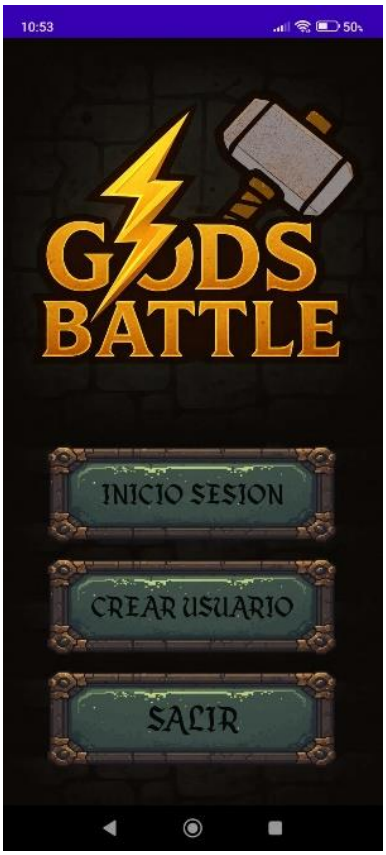
- Los administradores podrán consultar:
    - Estadísticas individuales de cada jugador: progreso, victorias, derrotas, objetos obtenidos, tiempo de juego.
    - Estadísticas globales: distribución de niveles, pisos alcanzados.
- Estas métricas permitirán ajustar el balance del juego y detectar patrones de comportamiento.

DESCRIPCIÓN FUNCIONAL DEL SISTEMA DE JUEGO



2 GESTIÓN DE PROYECTO

2.1 Introducción



Dentro de este apartado definiremos la planificación, organización, estimación, seguimiento y control del proyecto **“Gods Battle”**, desarrollado como aplicación móvil para dispositivos Android.

La Gestión de Proyecto se concibe un documento que podrá ser actualizado durante el desarrollo del proyecto en caso de que se produzcan desviaciones en la planificación inicial, modificaciones en el alcance o ajustes en los recursos asignados.

Este apartado permite estructurar el trabajo del equipo, distribuir responsabilidades, prever riesgos y establecer mecanismos de control que garanticen la correcta ejecución del proyecto.

2.2 Metodología de Trabajo

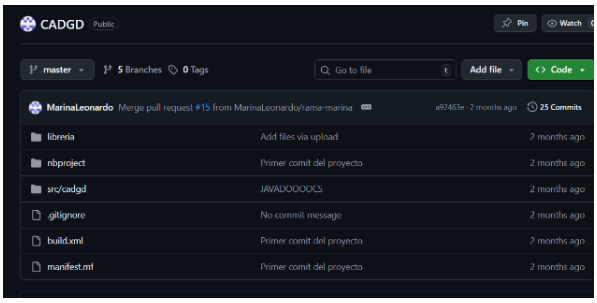
El desarrollo de Gods Battle seguirá una metodología híbrida que combina un enfoque estructurado por fases junto con un desarrollo iterativo en la fase de implementación.

El equipo estará formado por 4 desarrolladores, trabajando de manera colaborativa y con reparto de responsabilidades.

Metodología aplicada	
Aspecto	**Descripción**
Tipo de metodología	Híbrida (en cascada + iterativa)
Organización	Trabajo por fases + tareas
Control de versiones	GitHub
Gestión de tareas	Trello
Almacenamiento adicional	MEGA
Comunicación	Reuniones semanales + comunicación continua
Validación	Revisiones internas por fase

Prácticas de trabajo

- Reunión semanal de seguimiento
- Uso de repositorio Git para control de versiones
- Uso de Trello para organización de tareas
- Revisión interna antes de cerrar cada fase



- Trabajo colaborativo con reparto de responsabilidades

2.3 Fases del Proyecto

El proyecto se estructura en las siguientes fases:

1. **Objetivos y Alcance**

Definición del propósito del sistema y delimitación de sus funcionalidades.

2. **Análisis**

Identificación de requisitos funcionales, de interfaz, seguridad y sistema. Elaboración de diagramas de casos de uso y modelo entidad-relación.

3. **Diseño**

Definición de arquitectura del sistema, diagrama de clases y modelo relacional.

4. **Matrices de Trazabilidad**

Relación entre requisitos, modelos y diseño para garantizar coherencia.

5. **Implementación**

Desarrollo del sistema en Android Studio.

6. **Pruebas y Validación**

Verificación del correcto funcionamiento del sistema.

Las fases transversales (Gestión del Proyecto, Gestión de Configuración, Gestión de Riesgos y Aseguramiento de la Calidad) se desarrollarán durante todo el ciclo de vida del proyecto.

2.4 Planificación Temporal

La duración estimada del proyecto es de 12 semanas.

	Tiempo en semanas											
Fase	1	2	3	4	5	6	7	8	9	10	11	12
Objetivos y Alcance												
Análisis												
Diseño												
Matrices de trazabilidad												
Implementación												
Pruebas finales												

Las fases transversales abarcan desde la semana 1 hasta la semana 12.

2.5 Actividades y tareas

A continuación, se detallan las principales actividades y tareas del proyecto.

Fase	Actividad	Tarea	Responsable	Duración estimada
Análisis	Requisitos	Redacción requisitos funcionales	Equipo completo	4 días
Análisis	UML	Diagrama de casos de uso	Equipo completo	3 días
Diseño	Arquitectura	Diseño en capas	Equipo completo	2 días
Diseño	Clases	Diagrama de clases	Equipo completo	4 días
Implementación	Autenticación	Registro y login	Simón	5 días
Implementación	Combate	Lógica de turnos y daño	Marina	7 días
Implementación	Tienda	Compra y mejoras	Kevin	6 días
Implementación	Base de Datos	Modelo relacional en Data Modeler	Iván	6 días
Pruebas	Testing	Pruebas funcionales	Equipo completo	5 días

La asignación de responsables no impide la colaboración del resto del equipo en tareas complejas.

2.6 Recursos Necesarios

Recursos Humanos

- 4 desarrolladores junior.

Recursos Materiales

- Ordenadores personales.
- Android Studio.
- GitHub (repositorio privado).
- MEGA
- Trello
- Emuladores Android.
- Dispositivos Android físicos para pruebas.

Logística

- Reunión semanal de seguimiento.
- Comunicación continua mediante herramientas colaborativas.
- Entrega parcial en febrero y entrega final en junio.

2.7 Permisos y Autorizaciones

El proyecto no requiere permisos administrativos externos.

No obstante, se deberán cumplir las siguientes consideraciones:

- Protección de datos de los usuarios (almacenamiento cifrado de contraseñas).
- Cumplimiento de normativa académica para la presentación y defensa.
- Autorización del equipo docente para la defensa final.

2.8 Procedimiento de Ejecución de Tareas

Cada tarea seguirá el siguiente procedimiento:

1. Definición clara del objetivo de la tarea.
2. Desarrollo técnico.
3. Revisión interna por otro miembro del equipo.
4. Integración en la rama principal del repositorio.
5. Prueba funcional básica.
6. Cierre de tarea en el tablero Trello.

Este procedimiento permite minimizar errores y garantizar coherencia en el desarrollo.

2.9 Seguimiento y Control

Se establecerá un sistema de seguimiento continuo basado en:

Revisión Semanal

- Comparación entre tareas planificadas y tareas completadas.
- Detección de retrasos.
- Reasignación de tareas si es necesario.

Gestión de Cambios

En caso de modificación del alcance, tiempos o recursos:

1. Registro del cambio.
2. Evaluación del impacto.
3. Aprobación por consenso del equipo.
4. Actualización del documento de planificación.

Registro de Incidencias

- Uso de GitHub Issues.
- Clasificación por prioridad (Alta, Media, Baja).
- Asignación de responsable.
- Seguimiento hasta cierre.

	Tiempo en semanas											
Fase	1	2	3	4	5	6	7	8	9	10	11	12
Objetivos y Alcance												
Análisis												
Documentación												
Diseño												
Matrices de trazabilidad												
Implementación												
Pruebas finales												
Presentación												

2.10 Indicadores de Control

Para garantizar el correcto desarrollo del proyecto se establecen los siguientes indicadores:

Indicador	Método de Medición	Frecuencia
% tareas completadas	Tareas completadas / tareas planificadas	Semanal
Nº incidencias abiertas	Conteo en repositorio	Semanal
Nº bugs detectados	Registro en pruebas	Por sprint
Desviación temporal	Diferencia entre planificado y real	Semanal
% requisitos implementados	Requisitos implementados / totales	Final fase implementación

2.11 Participación en la evaluación

Para validar el producto desarrollado se realizarán:

- Pruebas funcionales realizadas por compañeros externos al equipo.
- Pruebas de usabilidad.
- Evaluación del manual de usuario.
- Revisión final por parte del equipo docente.

Los resultados de dichas evaluaciones podrán dar lugar a mejoras antes de la entrega final.

3 GESTIÓN DE LA CONFIGURACIÓN

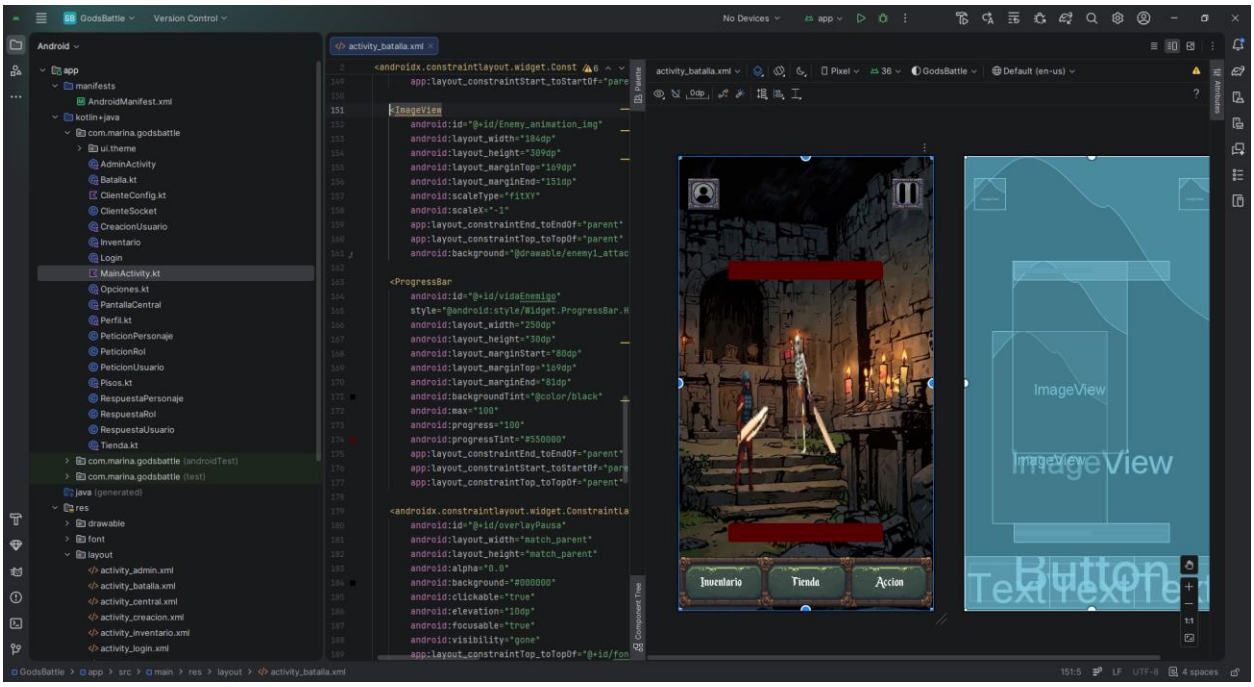
3.1 Introducción

El presente Plan de Gestión de la Configuración tiene como objetivo establecer los procedimientos necesarios para identificar, controlar, versionar y almacenar los Elementos de Configuración (EC) del proyecto, así como definir el proceso de gestión de incidencias que puedan producirse durante la ejecución del mismo.

Este plan garantiza la trazabilidad de los cambios, la integridad de los componentes del sistema y la correcta evolución del proyecto.

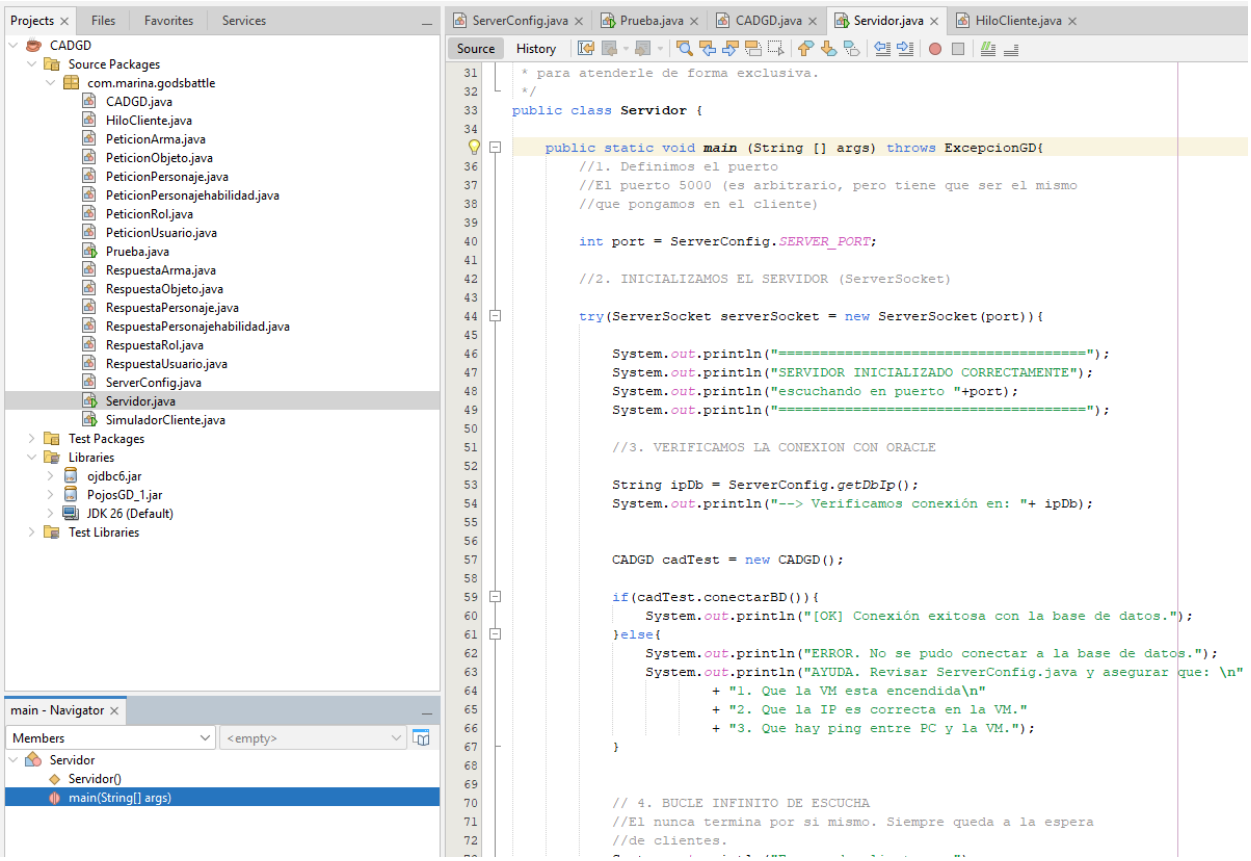
3.2 Aplicación Android

- Código fuente de la aplicación
- Actividades y fragmentos
- Clases de acceso a datos
- Clases POJO (modelos de datos)
- Clases de petición y respuesta
- Ficheros de configuración (Gradle, manifiesto, propiedades)
- Recursos XML (layouts, strings, estilos)
- Documentación técnica asociada



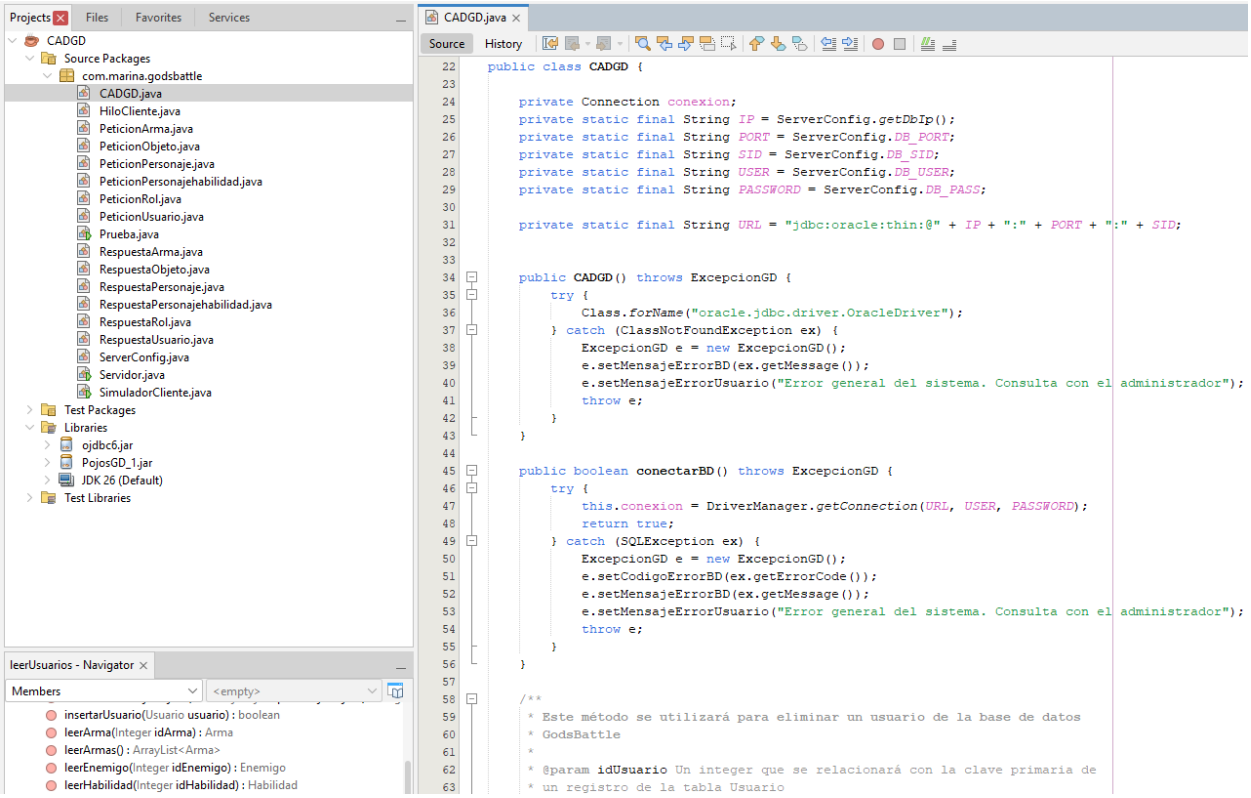
3.3 Servidor

- Código fuente del servidor
- Implementación del componente de acceso a datos
- Clases POJO
- Clases de petición y respuesta
- Controladores y servicios
- Ficheros de configuración del servidor
- Scripts de despliegue
- Documentación técnica del servidor



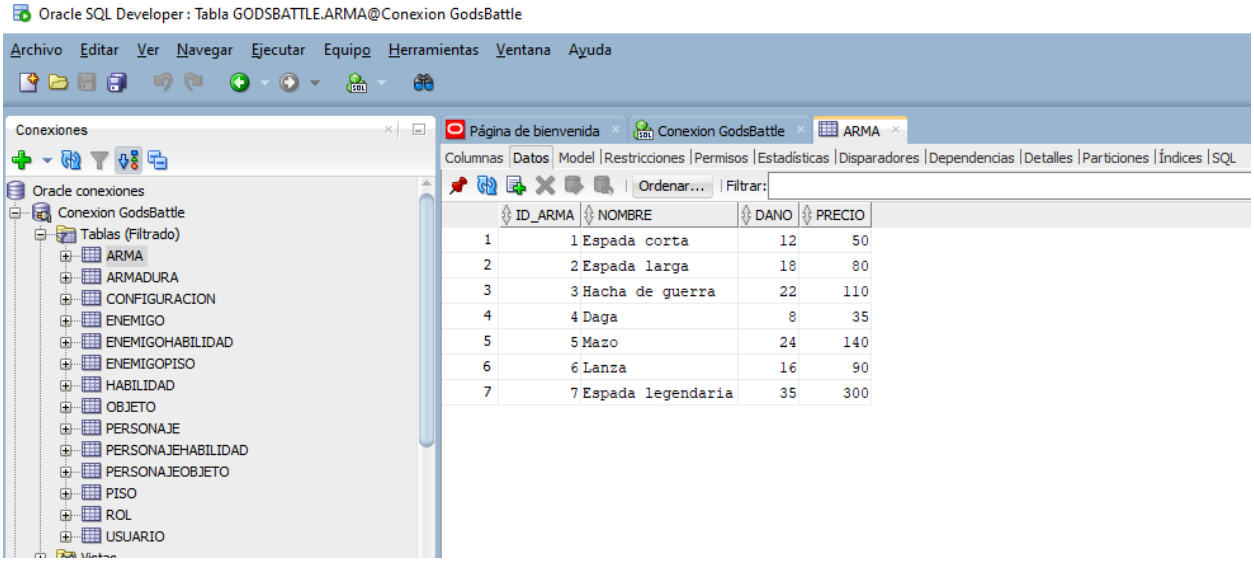
3.4 Componente de Acceso a Datos

- Interfaces de acceso a datos
- Implementaciones de acceso a base de datos
- Métodos CRUD
- Clases intermedias de transformación de datos



3.5 Base de Datos

- Scripts de creación de tablas (Create)
- Scripts de modificación de tablas (Delete)
- Scripts de carga inicial de datos (Insert)
- Procedimientos almacenados
- Secuencias
- Triggers
- Constraints
- Modelo entidad-relación (Data Modeler)



3.6 Documentación del Proyecto

- Especificación de requisitos
- Comentarios en código
- JavaDoc
- Documentación del proyecto

3.7 Nomenclatura de Versiones

Nombre_Día_Mes_Año

Ejemplo general:

- GodsBattle_18_02_2026
- Servidor_18_02_2026
- CAD_18_02_2026
- BD_18_02_2026

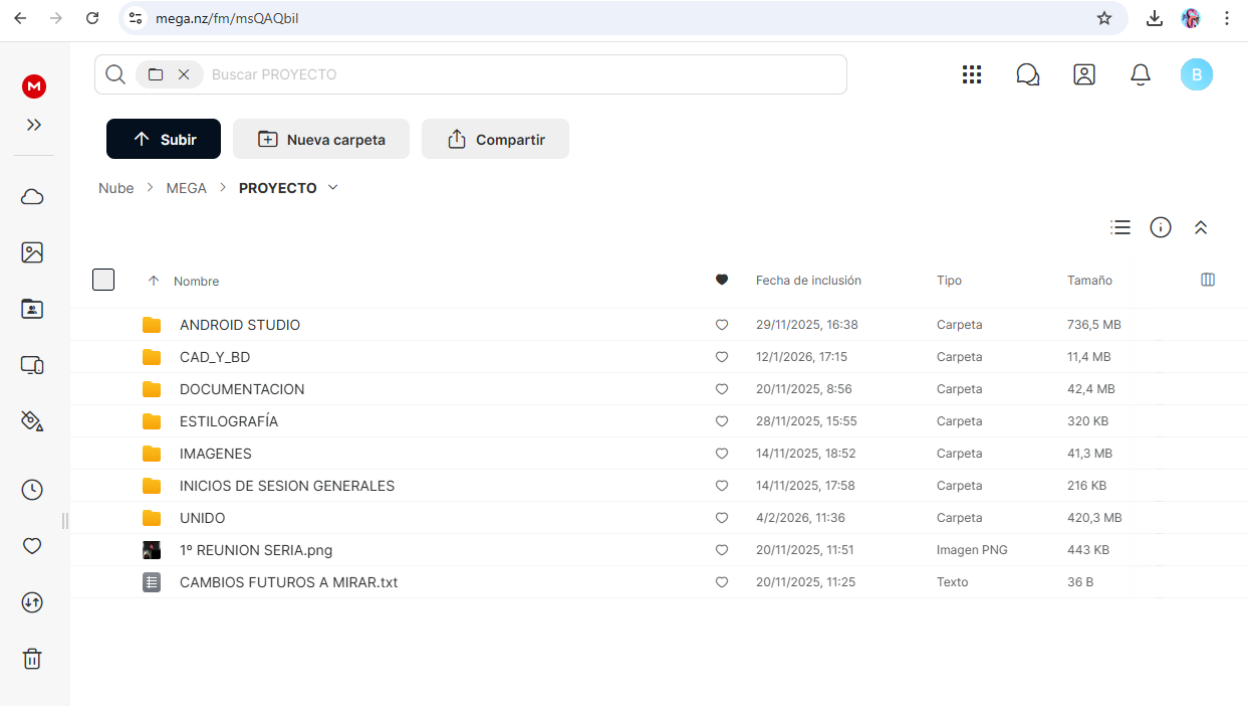
Si en un mismo día se generan varias versiones del mismo componente, se añadirá un número incremental al final:

- GodsBattle_18_02_2026_2
- GodsBattle_18_02_2026_3

3.8 Sistema de Almacenamiento

El proyecto utilizará un sistema de control de versiones alojado en:

- GitHub
- Mega



3.9 Metadatos de cada versión de EC

De cada versión generada se almacenarán los siguientes metadatos:

- Identificador de versión (formato fecha + nombre completo)
- Fecha y hora exacta de creación
- Autor del cambio
- Elemento de Configuración afectado
- Descripción detallada del cambio
- Incidencia asociada (si procede)

Estos metadatos se almacenarán en:

- El propio sistema Git (commits y etiquetas)
- Documentos de texto y comentarios en el código

3.10 Registro de la Incidencia

Toda incidencia deberá registrarse en un directorio específico.

El registro incluirá:

- Identificador único
- Fecha de detección
- Descripción detallada
- Pasos para reproducir el error
- Evidencias (logs, capturas, mensajes de error)
- Elemento de Configuración afectado
- Versión en la que se detecta

Ejemplo de identificador:

Incidencia_18_02_2026_01

3.11 Evaluación de la Incidencia

Para evaluar la incidencia se seguirán los siguientes pasos:

1. Analizar la reproducibilidad del problema.
2. Determinar el impacto funcional y técnico.
3. Identificar los EC afectados.
4. Decidir si se requiere una corrección inmediata o planificada.

La incidencia se clasificará como:

- Correctiva (error de funcionamiento)
- Evolutiva (nueva funcionalidad)
- Adaptativa (cambio por entorno externo)
- Perfectiva (mejora interna)

3.12 Solucionar la Incidencia

El proceso de resolución será el siguiente:

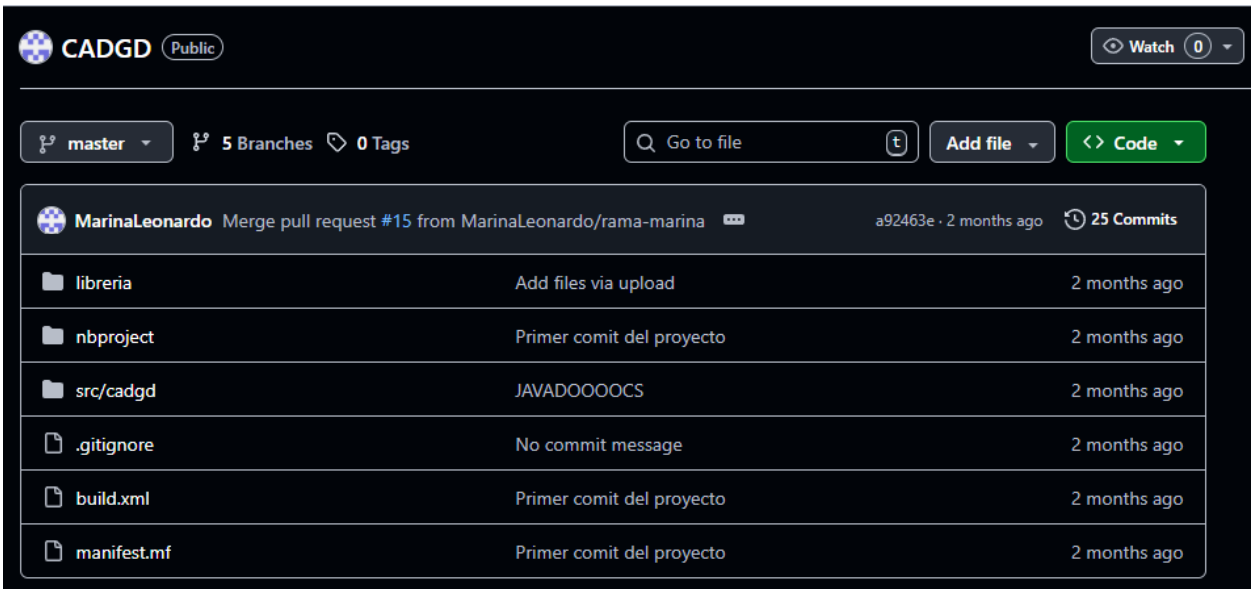
1. Creación de una rama específica para la corrección.
2. Modificación del EC afectado.
3. Actualización del código o scripts correspondientes.
4. Generación de nueva versión utilizando la fecha actual y el nombre completo del componente.
5. Actualización de los metadatos.
6. Realización de pruebas.
7. Validación que todo funcione correctamente.
8. Integración en la rama principal.
9. Despliegue en entorno correspondiente.
10. Cierre formal de la incidencia indicando la versión en la que se resolvió.

4 ANÁLISIS Y GESTIÓN DEL RIESGO

4.1 Introducción

El análisis y gestión del riesgo permite anticipar posibles problemas que puedan afectar al correcto funcionamiento del sistema, al cumplimiento de los requisitos establecidos y a la planificación del proyecto.

Este apartado tiene como objetivo identificar los principales riesgos asociados al desarrollo de un videojuego RPG para dispositivos Android, evaluar su probabilidad e impacto, y establecer medidas preventivas que permitan minimizar sus consecuencias durante el ciclo de vida del proyecto.



4.2 Identificación de Riesgos

Durante el análisis del proyecto se han detectado diferentes tipos de riesgos, que se agrupan en riesgos técnicos, funcionales, de seguridad y organizativos.

4.2.1 Riesgos Técnicos

Los riesgos técnicos están relacionados con la implementación y el funcionamiento interno del sistema:

- **Errores en el sistema de combate:** dado que el juego se basa en un sistema de combate por turnos con cálculos de daño, defensa, críticos y habilidades, existe el riesgo de que se produzcan errores en las fórmulas o en su aplicación, afectando al equilibrio del juego.
- **Problemas en la persistencia de datos:** la pérdida o corrupción de datos del jugador (progreso, inventario, dinero o habilidades) podría generar una experiencia negativa y pérdida de información relevante.
- **Rendimiento insuficiente:** el juego puede presentar problemas de rendimiento en dispositivos Android de gama media o baja, provocando bloqueos o una experiencia de usuario deficiente.
- **Fallos en la autenticación:** errores en el inicio de sesión o en la asignación de roles podrían impedir el acceso al juego o permitir accesos no autorizados.

4.2.2 Riesgos Funcionales

Los riesgos funcionales están relacionados con el cumplimiento de los requisitos definidos en la especificación:

- **Incumplimiento de requisitos funcionales:** existe el riesgo de que alguna funcionalidad definida no se implemente correctamente o quede incompleta por limitaciones de tiempo.

- **Errores en la progresión del juego:** fallos en el sistema de pisos podrían permitir el acceso a niveles no desbloqueados o bloquear el progreso del jugador.
- **Gestión incorrecta del inventario:** errores al usar, eliminar o consumir objetos podrían provocar inconsistencias en los datos del personaje.
- **Interfaz poco intuitiva:** una mala organización de las pantallas o de los menús puede dificultar la comprensión del juego por parte del usuario.

4.2.3 Riesgos de Seguridad

Los riesgos de seguridad afectan a la protección de los datos del sistema y de los usuarios:

- **Almacenamiento inseguro de contraseñas:** guardar contraseñas sin cifrado supone un riesgo para la seguridad de los usuarios.
- **Acceso indebido a funciones de administrador:** una gestión incorrecta de permisos podría permitir que jugadores accedan a estadísticas o funciones reservadas.
- **Manipulación de datos:** la modificación no autorizada de valores como dinero, estadísticas o inventario puede afectar al equilibrio del juego.

4.2.4 Riesgos Organizativos

Los riesgos organizativos están relacionados con la planificación y el trabajo en equipo:

- **Mala planificación del tiempo:** retrasos en el desarrollo por una estimación incorrecta de las tareas.
- **Falta de coordinación:** diferencias en los criterios de implementación entre los miembros del equipo.
- **Sobrecarga de trabajo:** acumulación de tareas en uno o varios integrantes del grupo.

4.3 Evaluación de Riesgos

Una vez identificados los riesgos, se procede a su evaluación según dos criterios: la probabilidad de que ocurran y el impacto que tendrían sobre el proyecto.

Riesgo	**Probabilidad**	**Impacto**	**Nivel**
Errores en el sistema de combate	Media	Alta	Alto
Pérdida de datos del jugador	Media	Alta	Alto
Problemas de rendimiento	Baja	Media	Medio
Fallos de autenticación	Media	Alta	Alto
Incumplimiento de requisitos	Media	Alta	Alto
Problemas organizativos	Media	Media	Medio

Los riesgos clasificados como **altos** requieren una mayor atención durante el desarrollo del proyecto.

4.4 Gestión de Riesgos

La gestión de riesgos consiste en definir un conjunto de medidas preventivas y organizativas destinadas a reducir la probabilidad de aparición de los riesgos identificados y, en caso de que se produzcan, minimizar su impacto sobre el desarrollo del proyecto *GodsBattle*.

En este proyecto, la gestión de riesgos se centra principalmente en la anticipación de problemas durante el desarrollo, evitando que estos se detecten únicamente en fases finales, donde su corrección supondría un mayor coste en tiempo y esfuerzo.

Para ello, se adoptan las siguientes medidas generales:

- Revisión continua del progreso del proyecto, verificando que las funcionalidades desarrolladas se ajustan a los requisitos definidos en la especificación.
- Control del cumplimiento de los requisitos funcionales, asegurando que cada módulo implementado corresponde a una funcionalidad previamente definida.
- Priorización de riesgos críticos, prestando especial atención a aquellos riesgos con mayor impacto, como la pérdida de datos del jugador o los errores en el sistema de combate.
- Organización y reparto equilibrado de tareas, con el fin de evitar sobrecargas de trabajo y reducir la probabilidad de errores.
- Prevención de errores de seguridad, mediante la validación de accesos y la correcta gestión de los roles de usuario.
- Aplicación de buenas prácticas de desarrollo, fomentando un código claro y estructurado que facilite la detección de errores.

La gestión de riesgos no se basa en la aplicación de soluciones técnicas concretas, sino en la adopción de estrategias generales que permitan mantener el control del proyecto durante todo su desarrollo.

4.5 Seguimiento del Riesgo

El seguimiento del riesgo es un proceso continuo que se realiza a lo largo de todo el ciclo de vida del proyecto *GodsBattle*. Su objetivo es comprobar que los riesgos identificados se mantienen bajo control y detectar la aparición de nuevos riesgos derivados de la evolución del proyecto.

Durante el desarrollo, se llevará a cabo un seguimiento periódico que permitirá:

- Revisar el estado de los riesgos previamente identificados, evaluando si su probabilidad o impacto ha cambiado.
- Detectar nuevos riesgos que puedan surgir como consecuencia de cambios en los requisitos o en la planificación.
- Comprobar la efectividad de las medidas preventivas aplicadas en la gestión del riesgo.
- Actualizar las prioridades, dando preferencia a los riesgos de nivel alto frente a los de nivel medio o bajo.
- Registrar incidencias relevantes, con el fin de mejorar la toma de decisiones durante el desarrollo.

Este seguimiento continuo permite actuar de forma temprana ante posibles problemas, reduciendo el impacto negativo sobre el proyecto y contribuyendo a una mejora progresiva de la calidad del sistema desarrollado.

5 ASEGURAMIENTO DE LA CALIDAD

5.1 Introducción

El presente Plan de Aseguramiento de la Calidad define las actividades, procedimientos, indicadores y mecanismos de evaluación necesarios para garantizar que el proyecto **GodsBattle RPG** cumple los requisitos funcionales, técnicos, de seguridad y de usabilidad establecidos en la Especificación de Requisitos.

El aseguramiento de la calidad tiene como finalidad:

- Garantizar el cumplimiento de los requisitos definidos.
- Detectar errores en fases tempranas del desarrollo.
- Asegurar la coherencia entre documentación, diseño e implementación.
- Validar la satisfacción del usuario final.

5.2 Objetivos del Aseguramiento de la Calidad

Asegurar que el 100% de los requisitos funcionales estén implementados correctamente y que el producto final no presente errores críticos.

Objetivos específicos:

1. Verificar la correcta implementación de los requisitos funcionales (RF).
2. Garantizar la integridad y seguridad de los datos.
3. Asegurar la estabilidad y rendimiento de la aplicación.
4. Validar la usabilidad del sistema mediante pruebas con usuarios.
5. Comprobar la calidad y claridad del manual de usuario.

5.3 Productos que se deben controlar

Los productos sometidos a control de calidad coinciden con los elementos definidos en la gestión de configuración del proyecto:

1. Especificación de Requisitos
2. Código Fuente (CAD, pojos…)
3. Base de Datos
4. Aplicación Ejecutable (APK)
5. Servidor
6. Manual de Usuario
7. Documentación Técnica (diagramas, modelos, diseño CAD)

5.4 Control de Calidad por Producto

5.4.1 Especificación de Requisitos

Indicadores de calidad

- Correctitud: requisitos definidos sin ambigüedades.
- Completitud: todos los requisitos funcionales y no funcionales definidos.
- Trazabilidad: requisitos numerados y clasificados (RF, RC, RS, RI).
- Consistencia: ausencia de contradicciones.

Procedimiento de evaluación

- Revisión cruzada entre miembros del equipo.
- Uso de lista de verificación:
    - ¿Todos los requisitos están numerados?
    - ¿Todos los campos incluyen tipo y restricciones?
    - ¿Están definidos claramente roles y entidades?

5.4.2 Código Fuente

Indicadores de calidad

- Cumplimiento de los requisitos funcionales.
- Ausencia de errores críticos.
- Modularidad y organización por capas.
- Buenas prácticas de programación.
- Legibilidad del código.

Procedimiento de evaluación

- Revisión de código entre compañeros.
- Pruebas unitarias de:
  - Cálculo de daño.
  - Sistema de turnos.
  - Gestión de inventario.
- Pruebas de integración:
  - Registro → creación de personaje.
  - Combate → desbloqueo de piso.
- Verificación antes de cada versión estable.

5.4.3 Base de Datos

Indicadores de calidad

- Integridad referencial.
- Restricciones correctamente aplicadas.
- No existencia de valores negativos donde no corresponda.
- Seguridad en almacenamiento de contraseñas (cifrado).
- Correctas relaciones entre entidades.

```
Output
CADGD (run)   CADGD (run) #2
run:
=====================================
SERVIDOR INICIALIZADO CORRECTAMENTE
escuchando en puerto 5000
=====================================
--> Verificamos conexión en: 172.20.10.10
[OK] Conexión exitosa con la base de datos.
Esperando clientes...
```

```
Source   History
/*
 * To change this license header, choose License Headers in Project Properties.
 * To change this template file, choose Tools | Templates
 * and open the template in the editor.
 */
package com.marina.godsbattle;

import pojosgd.ExcepcionGD;

/**
 *
 * @author keval
 */
public class Prueba {
    public static void main (String [] args) throws ExcepcionGD{


        CADGD prueba1 = new CADGD();
        System.out.println(prueba1.leerArmas());
    }
}
```

```
Output
CADGD (run)   CADGD (run) #2
run:
[Arma{armaId=1, armaNombre=Espada corta, armaDano=12, armaPrecio=50}, Arma{armaId=2, armaNombre=Espada larga, armaDano=18, armaPrecio=80}, Arma{armaId=3, armaNombre=Hacha de guerra, arma
BUILD SUCCESSFUL (total time: 1 second)
```

```
select * from arma
```

Resultado de la Consulta

SQL | Todas las Filas Recuperadas: 7 en 0,042 se

	ID_ARMA	NOMBRE	DANO	PRECIO
1	1	Espada corta	12	50
2	2	Espada larga	18	80
3	3	Hacha de guerra	22	110
4	4	Daga	8	35
5	5	Mazo	24	140
6	6	Lanza	16	90
7	7	Espada legendaria	35	300

Procedimiento de evaluación

- Pruebas con datos inválidos.
- Verificación de claves primarias y foráneas.
- Pruebas de autenticación.
- Revisión manual del esquema de base de datos.

5.4.4 Aplicación Ejecutable (APK)

Indicadores de calidad

- Funcionamiento estable sin cierres inesperados.
- Rendimiento fluido.
- Compatibilidad con dispositivos Android.
- Cumplimiento de requisitos de interfaz.
- Correcto funcionamiento del sistema de combate.

Procedimiento de evaluación

- Pruebas funcionales completas:
    - Registro e inicio de sesión.
    - Compra en tienda.
    - Uso de objetos.
    - Sistema de combate.
    - Subida y bajada de pisos.
- Pruebas en:
    - Emulador Android.
    - Dispositivo físico.

- Test de múltiples combates consecutivos.
- Registro de incidencias detectadas.

```
val peticion = Peticion( tipoOperacion = Peticion.TipoOperacion.READ, idPersonaje = 1)
```

Página de bienvenida | Usuario GodsBattle de SE de Kevin.sql | Usuario GodsBattle de SE de Kevin | PERSONAJE

Columnas | Datos | Model | Restricciones | Permisos | Estadísticas | Disparadores | Dependencias | Detalles | Particiones | Índices | SQL

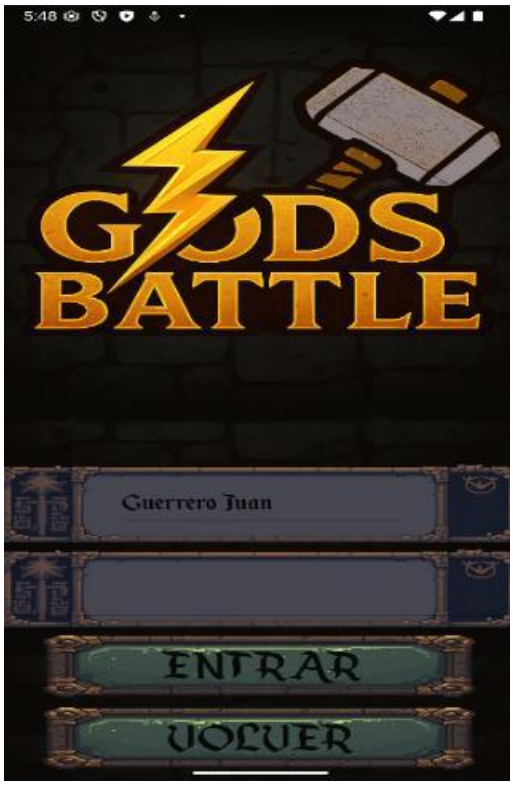
Ordenar... | Filtrar:

	ID_USUARIO	NOMBRE	VIDA_MAXIMA	DANO_BASE	DEFENSA_BASE	DINERO	ID_ARMA	ID_ARMADURA	ID_PISO
1	1	Guerrero Juan	100	12	6	150	1	2	1
2	2	Maga Ana	80	15	4	200	4	6	1

Como podemos observar, coincide el ID con el nombre del personaje, que en este caso se llama Guerrero Juan con unas estadísticas asociadas en nuestra base de datos.

Una vez asociado nuestro usuario, pincharemos en el botón entrar y procederemos a navegar por el juego para jugar o bien editar nuestras armas o pociones.

A continuación, una vez logueado podremos entrar a la pantalla de batalla donde decidiremos si equiparnos con mejor equipamiento tanto en la tienda como en el inventario.



5.4.5 Servidor

Indicadores de calidad

- Disponibilidad del servicio.
- Correcta autenticación.
- Seguridad en transmisión de datos.
- Integridad en almacenamiento de estadísticas.

Procedimiento de evaluación

- Pruebas de conexión.
- Simulación de múltiples accesos.
- Verificación de almacenamiento correcto de datos.

Como podemos observar, el color del cuadrado de conexión de la parte superior de nuestra aplicación GodsBattle es de color verde por lo que la conexión es exitosa.

En caso de que dicho cuadrado figurase en color rojo, nos mostraría que no estaría conectado.

5.4.6 Manual de Usuario

Indicadores de calidad

- Claridad del lenguaje.
- Cobertura de todas las funcionalidades.
- Inclusión de capturas de pantalla.
- Facilidad de comprensión.

Procedimiento de evaluación

- Revisión por un miembro diferente al autor.
- Entrega a usuario externo para prueba.
- Corrección según feedback recibido.

5.4.7 Documentación Técnica

Indicadores de calidad

- Coherencia con la implementación final.
- Diagramas claros y correctos.
- Explicación de arquitectura.
- Actualización tras cambios.

Procedimiento de evaluación

- Comparación con el código final.
- Revisión conjunta del equipo.
- Validación antes de la entrega final.

5.5 Procedimiento de Participación de los Usuarios

La evaluación del proyecto incluirá la participación de usuarios externos al equipo de desarrollo.

5.5.1 Evaluación del Producto Software

Procedimiento

1. Selección de usuarios de prueba.
2. Entrega del APK y manual.
3. Sesión de uso libre (30–45 minutos).
4. Cumplimentación de cuestionario estructurado.

Documento de Evaluación del Software

Cuestionario:

1. ¿El registro e inicio de sesión funcionan correctamente?
2. ¿El combate es fácil de entender? (1–5)
3. ¿La interfaz es intuitiva? (1–5)
4. ¿Has encontrado errores?
5. ¿La dificultad es adecuada?
6. Nivel de satisfacción general (1–5).

Indicadores medidos

- Media de satisfacción ≥ 3,5/5.
- Número de errores críticos detectados.
- Porcentaje de funcionalidades comprendidas sin ayuda.

5.5.2 Evaluación del Manual de Usuario

Procedimiento

1. Entregar únicamente el manual.
2. Pedir al usuario que:
    - Se registre.
    - Compre un objeto.
    - Inicie un combate.
3. Observar si necesita ayuda externa.

Documento de Evaluación del Manual

1. ¿El manual explica claramente cómo registrarse?
2. ¿Describe correctamente el sistema de combate?
3. ¿Te resultó fácil seguir los pasos? (1–5)
4. ¿Qué mejorarías?

Indicadores

- Porcentaje de tareas realizadas sin ayuda ≥ 80%.
- Nivel medio de comprensión ≥ 4/5.
- Número de dudas detectadas.

5.6 Gestión de Incidencias

Todas las incidencias detectadas durante las pruebas serán registradas siguiendo las reglas del punto 3.10 del documento.

5.7 Criterios de Aceptación Final

El proyecto será considerado apto para su entrega cuando:

- El 100% de los requisitos funcionales estén implementados.
- No existan errores críticos.
- La aplicación funcione sin cierres inesperados.
- La media de satisfacción de usuarios sea ≥ 3,5/5.
- El manual permita el uso autónomo del sistema.
- La documentación técnica sea coherente con la implementación final.

6 ANÁLISIS

6.1 Especificación de Requisitos

6.1.1 REQUISITOS FUNCIONALES

Definiciones

- **Usuario**: persona registrada en el sistema. Se dividen en:
    - **Jugador**: usuario con rol estándar que controla un personaje.
    - **Administrador**: usuario con permisos especiales para consultar estadísticas.
- **Personaje**: entidad controlada por el jugador en el juego.
- **Enemigo**: entidad hostil generada por el sistema.
- **Pisos**: niveles o escenarios del juego con enemigos asignados.
- **Objeto**: Consumible utilizado por el personaje.
- **Habilidad**: poderes especiales que ayudaran al personaje a ganar encuentros.

Gestión de Usuarios

RF.1 Registro de usuario

El sistema debe permitir registrar un usuario con:

- Nombre: Tipo textual de longitud (50). (Obligatorio)
- Correo: Tipo textual de longitud (50). (Obligatorio) (Seguirá el patrón de un correo)
- Contraseña: Tipo textual de longitud (300) y cifrado. (Obligatorio)
- Rol (Jugador o Administrador), de base el ROL será jugador: Tipo numérico entero que hace referencia al tipo de usuario. (Obligatorio)

RF.2 Inicio de sesión y registro

El usuario iniciará sesión o se registrará nada más entrar en la aplicación.

De forma predeterminada será un usuario normal, pero un administrador podrá dar privilegios de administrador.

RF.3 Gestión del perfil

- El usuario podrá modificar su nombre y contraseña.
- Solo el administrador podrá cambiar roles.

RF.4 Asociación de personaje

Cada usuario debe estar vinculado a **exactamente un personaje**.

Gestión de Personaje

RF.5 Creación del personaje

Al registrarse, el jugador creará un personaje con:

- Nombre, a elección del usuario: Tipo textual de longitud (50). (Obligatorio)
- Puntos de vida máximos inicial, predeterminado por el sistema: Tipo numérico entero de longitud (4) (Obligatorio)
- Daño base, predeterminado por el sistema: Tipo numérico entero de longitud (4) (Obligatorio)
- Defensa base, predeterminado por el sistema: Tipo numérico entero de longitud (4) (Obligatorio)
- Dinero, predeterminado por el sistema: Tipo numérico entero de longitud (4) (Obligatorio)

RF.6 Equipamiento

El personaje puede tener cierto equipamiento.

- 1 arma
- 1 armadura

RF.7 Inventario

El personaje podrá almacenar, usar y tirar objetos de su inventario.

RF.8 Habilidades del personaje

El personaje podrá tener entre 0 y 3 habilidades que se obtendrán mediante la compra de pergaminos en la tienda.

RF.9 Progresión

El personaje puede subir y bajar por los pisos, siendo el último piso vencido lo máximo que puede acceder para conseguir dinero y comprar armas, armaduras, objetos y habilidades.

Además, podrá mejorar su daño base, defensa base y vida máxima desde la tienda.

Gestión de Enemigos

RF.10 Datos del enemigo

Un enemigo tendrá:

- Nombre: Tipo textual de longitud (50). (Obligatorio)
- Puntos de vida máximos: Tipo numérico entero de longitud (4) (Obligatorio) (No puede ser negativo)
- Daño base: Tipo numérico entero de longitud (4) (Obligatorio) (No puede ser negativo)
- Defensa base: Tipo numérico entero de longitud (4) (Obligatorio) (No puede ser negativo)
- Dinero otorgado: Tipo numérico entero de longitud (4) (Obligatorio) (No puede ser negativo)

RF.11 Habilidades del enemigo

Un enemigo podrá tener entre 0 y 2 habilidades. Cuando el enemigo utilice una habilidad, esta se elegirá aleatoriamente en caso de tener 2.

En el caso de recibir un ataque crítico de parte del jugador, se eliminará aleatoriamente una de sus habilidades en el caso de tener.

RF.12 Enemigos por piso

Los enemigos estarán asociados a uno o varios pisos.

Gestión de Pisos

RF.13 Información del piso

Cada piso tendrá:

- Número: Tipo numérico entero de longitud (4) (Obligatorio)
- Background o imagen. Tipo textual de longitud (300). (Obligatorio)
- Lista de enemigos asociados al piso seleccionado/actual.

Se guardará el número del piso más elevado al que haya alcanzado un personaje para impedir la entrada a este si no se ha vencido el anterior. El piso 1 siempre estará abierto al ser el primero.

Objetos

RF.14 Gestión de objetos

Cada objeto tiene:

- Nombre: Tipo textual de longitud (50). (Obligatorio)
- Descripcion (curación, ventajas (o también llamado buffs), desventajas (o también llamado debuffs)): Tipo textual de longitud (200). (Obligatorio)
- Precio, coste del objeto. Tipo numérico entero de longitud (4) (Obligatorio)
- Cada objeto tendrá una cierta cantidad de usos: Tipo numérico entero de longitud (4) (Obligatorio) (No puede ser negativo)

RF.15 Asignación de objetos

Los objetos pertenecen al personaje a través de su inventario.

Estos pueden ser comprados en la tienda a través del dinero recibido de los enemigos derrotados

Armas y Armaduras

RF.16 Armas

Un arma tendrá:

- Nombre: Tipo textual de longitud (50). (Obligatorio)
- Estadísticas de ataque: Tipo numérico entero de longitud (4) (Obligatorio) (No puede ser negativo)
- Precio: Tipo numérico entero de longitud (4) (Obligatorio) (No puede ser negativo)

RF.17 Armaduras

Una armadura tendrá:

- Nombre: Tipo textual de longitud (50). (Obligatorio)
- Estadísticas de defensa: Tipo numérico entero de longitud (4) (Obligatorio) (No puede ser negativo)
- Precio: Tipo numérico entero de longitud (4) (Obligatorio) (No puede ser negativo)

RF.18 Asignación de objetos

Las armas y armaduras serán únicas para el personaje adquiridas solamente a través de la tienda.

En el primer combate no tendrá ni arma ni armadura.

Estos serán comprados en la tienda a través del dinero recibido de los enemigos derrotados.

Sistema de Combate por Turnos

RF.19 Iniciar combate

El personaje podrá iniciar un combate contra un enemigo disponible en el piso actual.

RF.20 Turnos

El sistema alternará turnos entre personaje y enemigo.

RF.21 Cálculo de daño y defensa

Para este juego, especificaremos diferentes fórmulas para establecer tanto el daño de ataque como el daño de defensa.

Para el daño del ataque, aplicaremos la siguiente fórmula que consta del daño base del personaje más la modificación del arma del personaje, más ventajas del personaje. Después este se restará por la defensa del enemigo.

El daño se calcula como:

DAÑO = ((DAÑO_BASE + MODIFICACION_ARMA) + VENTAJAS) – DEFENSA_ENEMIGO)

PROBABILIDAD_CRITICO = 10%

DAÑO_CRITICO = DAÑO * 1.3

Para el daño recibido, aplicaremos la siguiente fórmula que consta de la defensa base del personaje más la modificación de la armadura que podemos adquirir más ventajas de habilidades u objetos que podemos adquirir.

El daño recibido se calcula como:

DAÑO RECIBIDO = (DAÑO BASE ENEMIGO + VENTAJAS DEL ENEMIGO) – ((DEFENSA_BASE + MODIFICACION_ARMADURA) + VENTAJAS)

RF.22 Uso de habilidades

El personaje y el enemigo podrán usar habilidades durante el combate.

RF.23 Uso de objetos

El personaje podrá usar objetos de su inventario acabando así su turno.

En el caso de que dicho objeto tenga más de un uso, solamente bajará en 1 sus usos restantes, en el caso de acabarse, se eliminará del inventario.

RF.24 Finalización del combate

El combate termina cuando alguno llega a 0 de vida.

Si gana al enemigo, conseguirá el dinero que este tiene y se desbloqueará el siguiente piso en caso de no estarlo ya.

Si pierde, no conseguirá ninguna recompensa, y volverá a la pantalla de selección de pisos con su vida máxima.

Seguimiento del combate

RF.25 Registro de estados

El sistema almacenará:

- Último piso que haya alcanzado el personaje

RF.26 Salud

El personaje tendrá unos puntos de vida iniciales (VIDAMAXIMA).

Estos puntos de vida pueden mejorarse en la tienda.

La vida actual del personaje no puede sobrepasar su vida máxima. Esto puede ocurrir al utilizar un objeto que le cure.

(Cada vez que acabas la batalla, se reinicia la vida del personaje)

Funciones del Administrador

RF.27 Consultar estadísticas globales

El administrador podrá ver:

- Número total de usuarios
- Piso máximo desbloqueado por usuarios

RF.28 Supervisión

El administrador podrá revisar:

- Habilidades existentes
- Objetos disponibles
- Armas y armaduras

6.1.2 REQUISITOS DE CONFIGURACIÓN DEL SISTEMA

RC.1: Configuración básica del juego (volumen, idioma)

- Creación de una configuración del volumen
- Posibilidad de cambio de idioma a inglés o español

6.1.3 REQUISITOS DE SEGURIDAD

RS.1. Autenticación en el Sistema

Como autenticación en el sistema, el usuario entrará en la aplicación mediante su correo y su contraseña que se guardará en la propia base de datos, la cual estará cifrada.

6.1.4 REQUISITOS DE INTERFAZ DE USUARIO

RI.1. Aplicación móvil e interfaz

La aplicación será accesible mediante dispositivos Android.

RI.2. Dispositivos Soportados

- Dispositivos móviles, exceptuando IOS.
- Tabletas Android
- Emuladores Android

RI.3. Principios de Diseño

- Interfaz limpia y sencilla.
- Intuitiva.
- Coherencia entre todas las pantallas.
- Secciones claramente organizadas.

6.2 Estudio de Viabilidad Técnica

6.2.1 INTRODUCCIÓN A LA VIABILIDAD

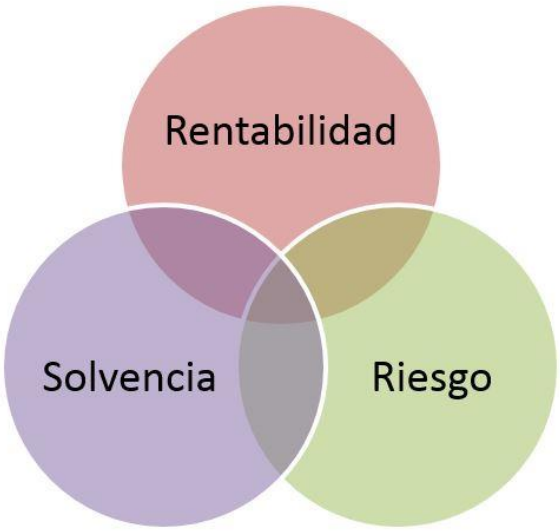
El análisis de viabilidad es un paso fundamental en el desarrollo de cualquier proyecto, ya que permite determinar si es posible llevarlo a cabo con los recursos disponibles y si tiene sentido tanto a nivel técnico como práctico. No se trata únicamente de verificar la capacidad de desarrollo del proyecto, sino también de evaluar su sostenibilidad, funcionamiento y mantenimiento a lo largo del tiempo.

GodsBattle es un videojuego RPG para dispositivos Android que combina combate por turnos, progresión del personaje y gestión de elementos como inventario, habilidades o equipamiento. Su desarrollo requiere integrar diferentes componentes del sistema, incluyendo la aplicación móvil, el servidor y la base de datos, lo que hace imprescindible analizar su viabilidad desde distintos enfoques: técnico, económico y operativo.

Desde el punto de vista técnico, el proyecto se apoya en tecnologías con las que el equipo tiene experiencia, como Android Studio con Kotlin y un servidor en Java conectado a base de datos Oracle. Esto facilita el desarrollo y reduce posibles riesgos. Además, el sistema está modularizado (usuarios, combate, tienda, inventario), lo que permite un mantenimiento más sencillo y facilita futuras ampliaciones.

En términos económicos, el proyecto es accesible, ya que se utilizan herramientas gratuitas y recursos propios, reduciendo costos iniciales. El modelo de negocio planteado es freemium, permitiendo que el juego sea gratuito, pero con compras opcionales y publicidad, una estrategia común en juegos móviles que facilita el alcance de un mayor número de usuarios.

Finalmente, la viabilidad operativa considera el uso práctico del sistema. *GodsBattle* está diseñado para ser intuitivo y fácil de usar, con pruebas en distintos dispositivos Android que garantizan un rendimiento estable y una experiencia satisfactoria para los usuarios.



6.2.2 MATRIZ DE VIABILIDAD

Para evaluar el proyecto de forma completa, es necesario analizar su viabilidad desde diferentes puntos de vista: técnico, económico y operativo.

Cada uno de estos aspectos permite comprobar si el proyecto puede desarrollarse correctamente, mantenerse en el tiempo y ser utilizado de forma adecuada por los usuarios.

A continuación, se presenta una matriz de viabilidad que resume estos factores, mostrando de forma clara la situación del proyecto y el nivel estimado en cada caso.

6.2.3 Viabilidad Técnica

Arquitectura del Sistema

El sistema de GodsBattle sigue una arquitectura cliente-servidor con tres capas principales:

1. Cliente móvil: Aplicación Android desarrollada en Kotlin.
2. Servidor intermedio: Implementado en Java, encargado de la lógica del juego y la comunicación con la base de datos.
3. Base de datos: Sistema Oracle para la gestión de usuarios, personajes, inventario y estadísticas.

Esta arquitectura permite una separación clara de responsabilidades, facilitando el mantenimiento, la escalabilidad y la evolución futura del sistema.

Tecnologías Utilizadas

- Android Studio + Kotlin: Desarrollo de la app móvil, ofreciendo concisión, seguridad frente a errores y compatibilidad completa con Android.
- Java (Servidor): Gestión de la lógica del servidor mediante sockets, manejando peticiones y respuestas del cliente.
- Base de datos Oracle: Almacenamiento estructurado de información relevante del juego.
- Comunicación por sockets: Permite el intercambio de objetos complejos entre cliente y servidor.
- Corrutinas en Kotlin: Facilitan la ejecución de tareas en segundo plano, evitando bloqueos y mejorando la experiencia del usuario.

Viabilidad del Desarrollo

El desarrollo del sistema es viable por varios motivos:

- Disponibilidad de herramientas: Todas las tecnologías utilizadas son accesibles y cuentan con amplia documentación.
- Compatibilidad multiplataforma: La aplicación funciona en una amplia variedad de dispositivos Android y tablets.
- Modularidad: Permite trabajar de forma independiente en cliente, servidor y base de datos.
- Reutilización de código: Archivos JAR compartidos entre cliente y servidor garantizan coherencia en la lógica del juego.

Rendimiento y Optimización

Se realizaron pruebas en distintos dispositivos Android, analizando:

- Uso de CPU
- Consumo de memoria
- Tráfico de red
- Acceso a disco

Los resultados muestran un rendimiento estable y eficiente, aunque se identificaron diferencias en la adaptación de la interfaz, indicando la necesidad de optimizar el diseño responsive.

Seguridad del Sistema

El sistema incorpora medidas básicas de seguridad:

- Autenticación de usuarios mediante correo y contraseña.
- Cifrado de contraseñas en la base de datos.
- Separación de roles (jugador y administrador).

Para versiones futuras, se recomienda implementar protocolos más avanzados como HTTPS o autenticación basada en tokens.

Limitaciones Técnicas

Algunas limitaciones identificadas incluyen:

- Uso de sockets frente a APIs REST, lo que puede afectar la escalabilidad en entornos complejos.
- Adaptación a diferentes resoluciones aún incompleta.
- Dependencia de un servidor propio, aumentando la gestión de despliegue y mantenimiento.

Viabilidad Económica

El proyecto es económicamente viable debido a:

- Bajo coste de desarrollo: Se utilizan herramientas gratuitas y recursos propios.
- Modelo de negocio escalable: Estrategia freemium que permite atraer usuarios gratuitamente y generar ingresos mediante compras internas y publicidad.
- Rentabilidad combinada: Algunos elementos generan ingresos directos, mientras que otros requieren estrategias combinadas para maximizar beneficios.

Se recomienda optimizar la estrategia de monetización para garantizar rentabilidad a largo plazo.

Viabilidad Operativa

La viabilidad operativa de GodsBattle es alta:

- La aplicación es intuitiva y accesible, adaptada a dispositivos móviles.
- Se planifican actualizaciones y mejoras continuas.
- Pruebas de rendimiento en distintos dispositivos muestran estabilidad y eficiencia.

Esto asegura que los usuarios puedan interactuar con el sistema de manera satisfactoria y que el proyecto pueda evolucionar de forma controlada.

6.2.4 Conclusión

El proyecto GodsBattle presenta una viabilidad integral alta, sustentada en:

- Tecnologías conocidas y consolidadas, accesibles y compatibles con Android.
- Arquitectura modular y escalable, que facilita el mantenimiento y la evolución del sistema.
- Bajo coste de desarrollo y modelo de negocio freemium, lo que permite atraer usuarios y generar ingresos.
- Diseño operativo intuitivo y rendimiento estable, garantizando una experiencia positiva para los usuarios.

Aunque se han identificado limitaciones técnicas y económicas, estas representan oportunidades de mejora y no comprometen el desarrollo del proyecto. Se recomienda optimizar el modelo de monetización, adaptar la interfaz a todas las resoluciones y considerar protocolos de seguridad más avanzados para futuras versiones.

6.3 Diagrama de Casos de Uso

El sistema *GodsBattle* se estructura en torno a dos actores principales: el **Jugador** y el **Administrador**, cada uno con diferentes niveles de interacción con el sistema.

Identificación de actores

- **Jugador**: usuario principal que interactúa con el videojuego y controla un personaje.
- **Administrador**: usuario con privilegios avanzados orientados a la supervisión y análisis del sistema.

6.3.1 Casos de uso del Jugador

El actor *Jugador* participa en los siguientes casos de uso:

Gestión de acceso

- Registrarse en el sistema.
- Iniciar sesión.
- Gestionar perfil (modificación de datos personales).

Gestión del personaje

- Crear personaje.
- Consultar estadísticas.
- Mejorar atributos (vida, daño, defensa).
- Equipar armas y armaduras.

Gestión de inventario

- Almacenar objetos.
- Usar objetos.
- Eliminar objetos.

Interacción con el sistema de juego

- Acceder a la tienda.
- Comprar objetos, armas, armaduras y habilidades.
- Iniciar combate.
- Realizar acciones en combate:
    - Atacar
    - Usar habilidades
    - Usar objetos

Progresión

- Avanzar por pisos.
- Repetir niveles para obtener recursos.

6.3.2 Casos de uso del Administrador

El actor *Administrador* dispone de funcionalidades orientadas al control del sistema:

- **Consultar estadísticas globales:**
    - Número de usuarios
    - Progreso de jugadores

- **Supervisar contenido del sistema:**
  - Habilidades
  - Objetos
  - Equipamiento

- **Gestionar usuarios:**
  - Modificación de roles

6.3.3 Relaciones entre casos de uso

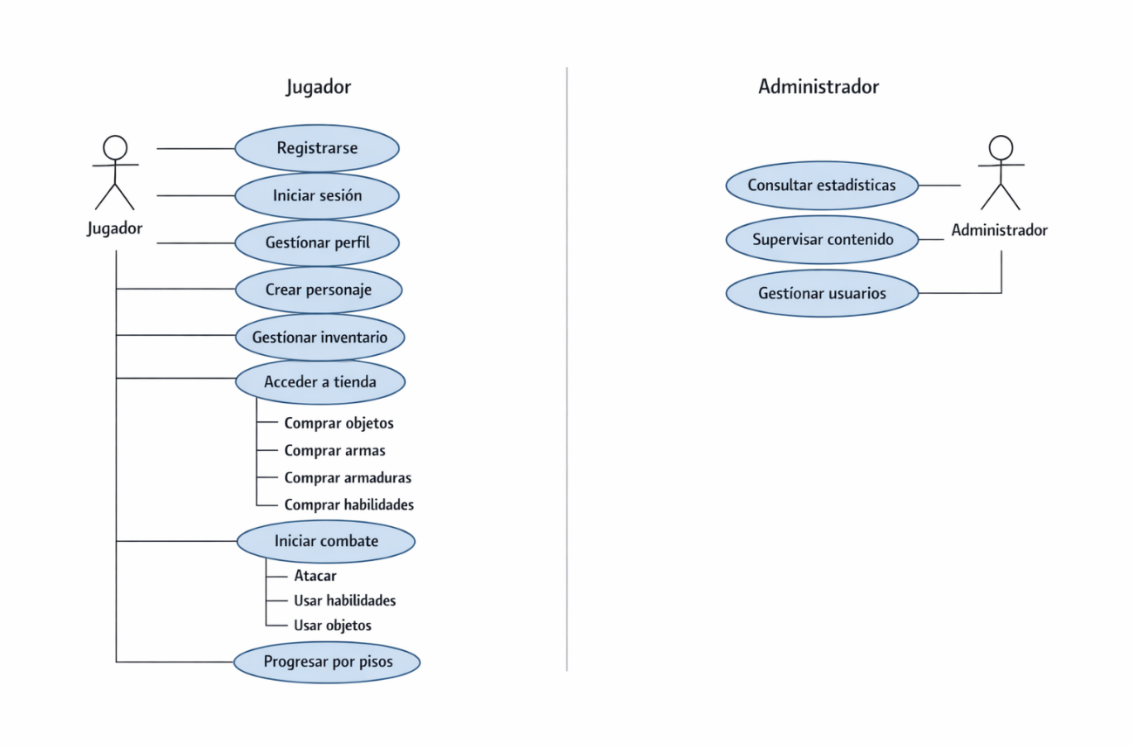
El sistema presenta las siguientes relaciones:

- **«include»**:
  - *Iniciar combate* incluye *Realizar acciones en combate*.
  - *Acceder a tienda* incluye *Comprar objetos/equipamiento*.
- **«extend»**:
  - *Usar habilidades* extiende el caso de uso *Combatir*.
  - *Usar objetos* extiende el sistema de combate.

Estas relaciones reflejan la estructura modular y reutilizable del sistema.

6.3.4 Representación estructurada del diagrama

A nivel conceptual, el diagrama de casos de uso se organiza de la siguiente manera:

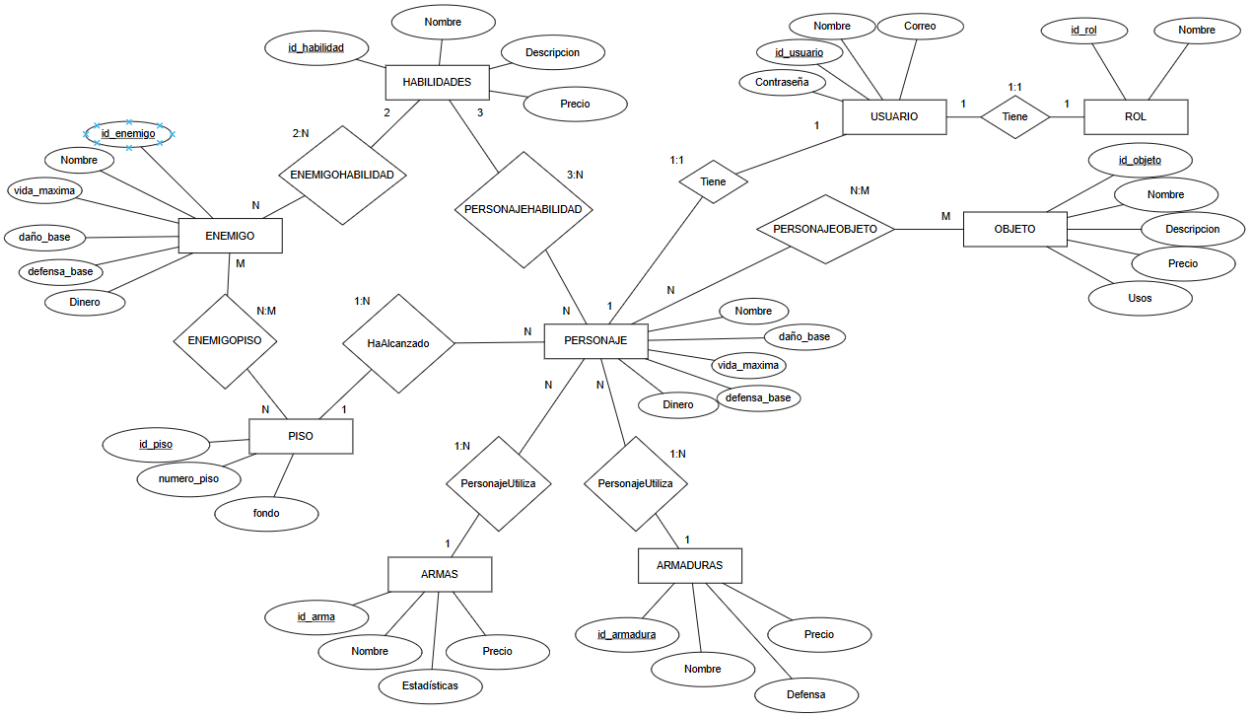


Interpretación del modelo

El modelo de casos de uso refleja claramente:

- Una **orientación centrada en el jugador**, donde la mayoría de las funcionalidades están enfocadas a la experiencia de juego.
- Una **separación clara de roles**, evitando conflictos entre funcionalidades de usuario y administración.
- Una estructura modular que facilita la ampliación futura del sistema.

6.4 Modelo Entidad Relación



Restricciones de negocio:

- **Tabla Usuario:** Se establece que el correo electrónico debe cumplir un formato básico válido, incluyendo un carácter "@" y un dominio.
- **Tabla Configuración:** La vida máxima, daño base, y defensa base predeterminada debe ser mayor o igual a cero.
- **Tabla Piso:** El número del piso debe ser mayor o igual a cero.
- **Tabla Arma:** El daño y el precio del arma debe ser mayor o igual a cero.
- **Tabla Armadura:** La defensa y el precio del arma debe ser mayor o igual a cero.
- **Tabla Objeto:** El precio y el número de usos del objeto debe ser mayor o igual a uno.
- **Tabla Habilidad:** El precio de la habilidad debe ser mayor o igual a cero.
- **Tabla Enemigo:** La vida máxima, daño base, defensa base y dinero del enemigo debe ser mayor o igual a cero.

- **Tabla Personaje:** La vida máxima debe ser mayor o igual a uno, y el daño base, defensa base y dinero del enemigo debe ser mayor o igual a cero.
- **Tabla PersonajeObjeto:** Los usos restantes de un objeto deben ser mayor o igual a cero. Cuando un registro tenga 0 usos restantes, se eliminará.

7 DISEÑO

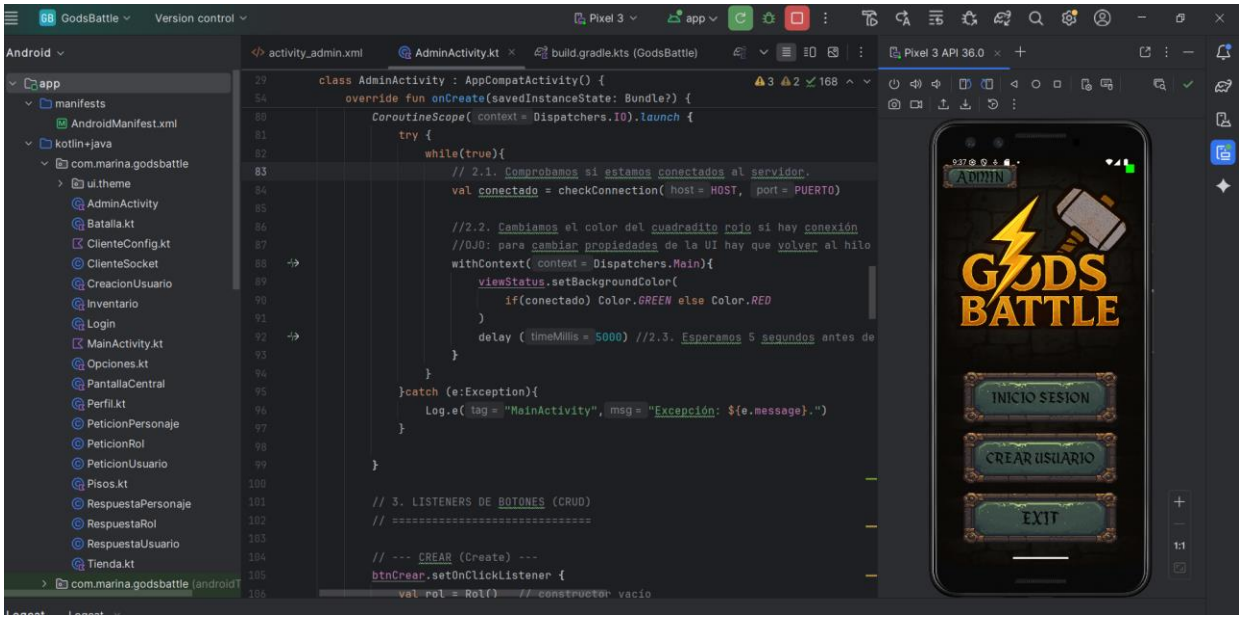
7.1 Arquitectura del Sistema

El sistema seguirá una arquitectura en capas para garantizar la separación de responsabilidades y facilitar el mantenimiento. Además, esta estructura facilita la modificación de esta para otros dispositivos.

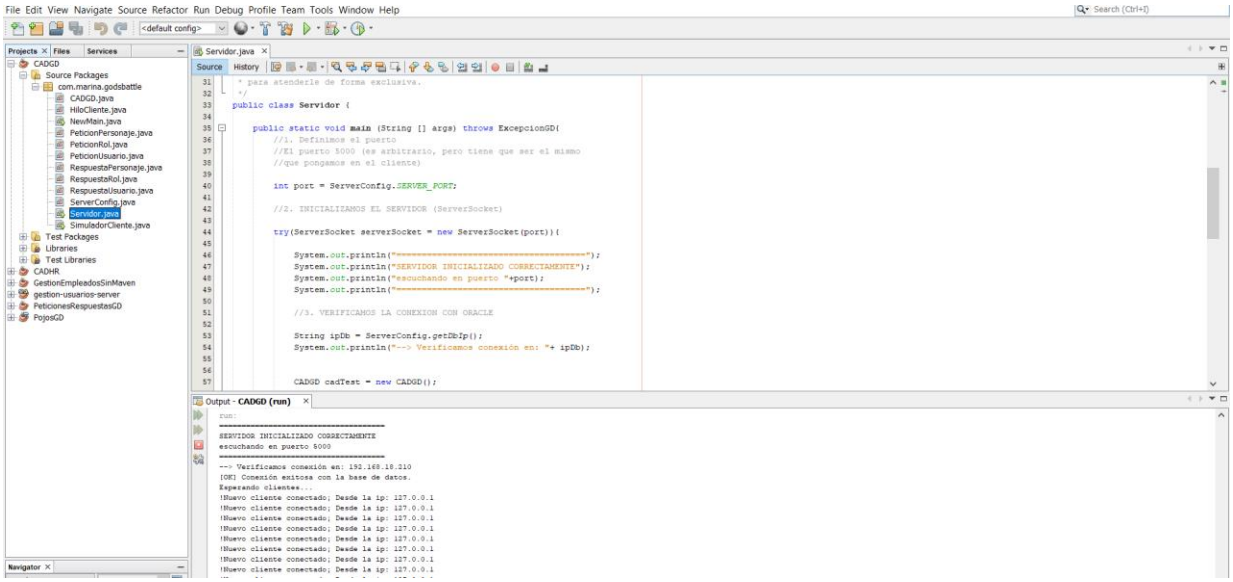
Componentes principales	
Aplicación Android (Frontend)	• Interfaz de usuario • Gestión de interacción con el jugador
Lógica de negocio	• Sistema de combate • Gestión de personajes • Sistema de tienda
Acceso a datos	• Comunicación con base de datos • Persistencia de datos
Base de datos	• Almacenamiento de usuarios, personajes, objetos y estadísticas

Interacción entre componentes

1. El usuario interactúa con la app (Gods Battle app)



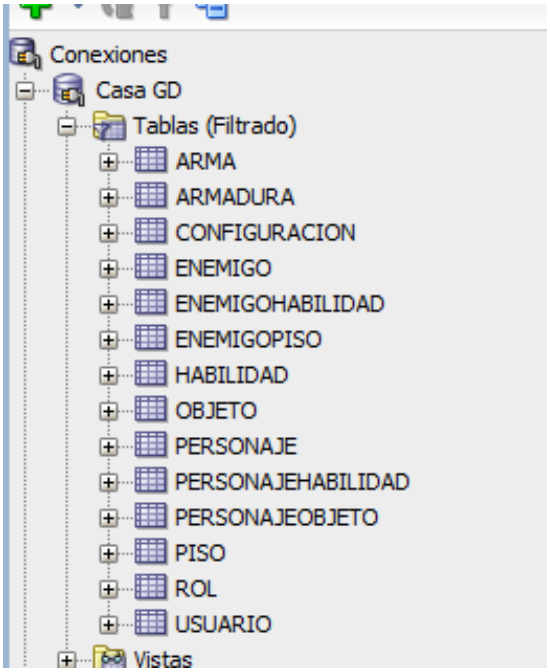
2. La app envía peticiones a la lógica de negocio (Logica del juego)



3. La lógica procesa datos y consulta la base de datos (CAD)

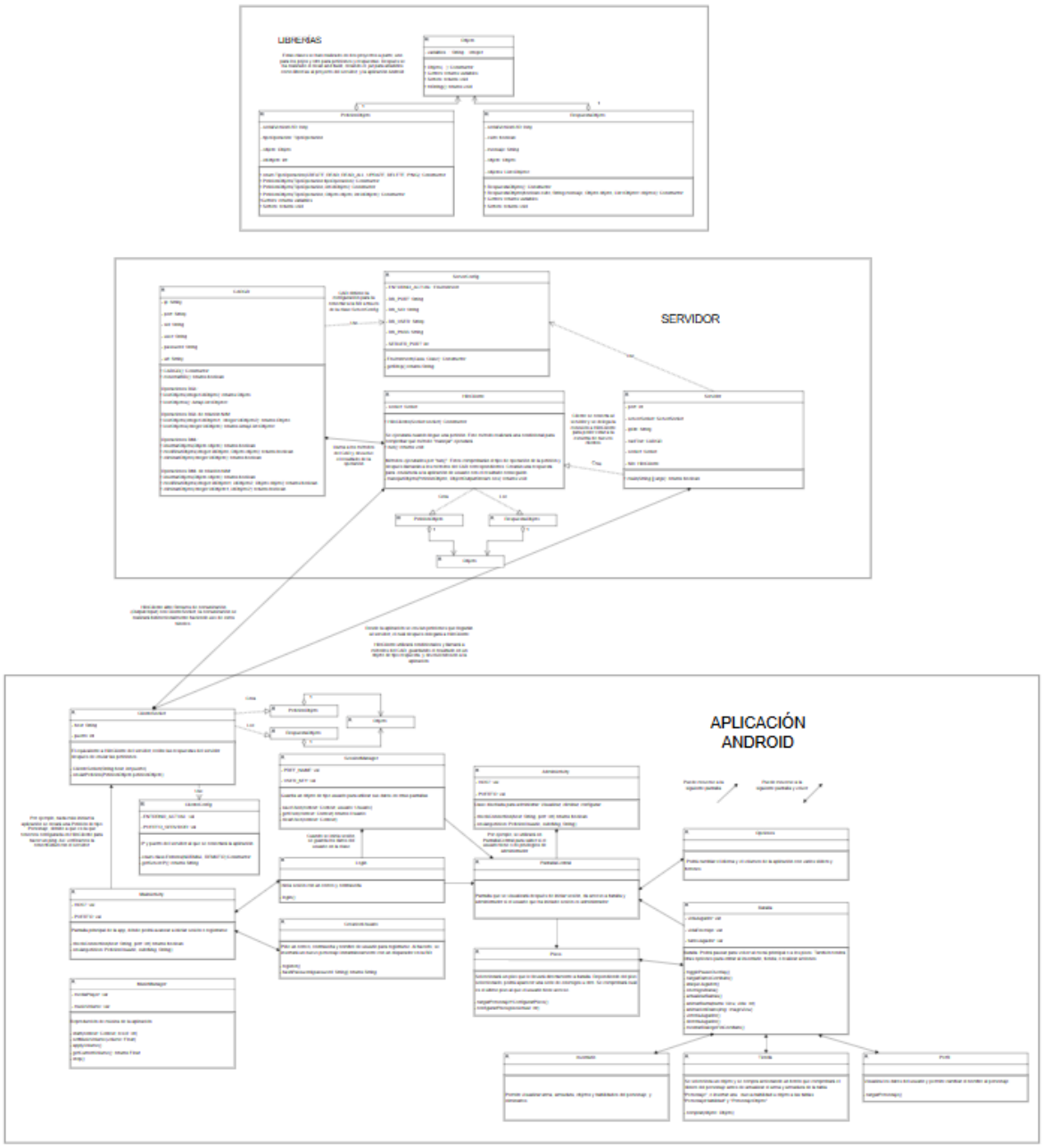
```
public class CADGD {

    private Connection conexion;
    private static final String IP = ServerConfig.getDbIp();
    private static final String PORT = ServerConfig.DB_PORT;
    private static final String SID = ServerConfig.DB_SID;
    private static final String USER = ServerConfig.DB_USER;
    private static final String PASSWORD = ServerConfig.DB_PASS;

    private static final String URL = "jdbc:oracle:thin:@" + IP + ":" + PORT + ":" + SID;

    public CADGD() throws ExcepcionGD {
        try {
            Class.forName("oracle.jdbc.driver.OracleDriver");
        } catch (ClassNotFoundException ex) {
            ExcepcionGD e = new ExcepcionGD();
            e.setMensajeErrorBD(ex.getMessage());
            e.setMensajeErrorUsuario("Error general del sistema. Consulta con el administrador");
            throw e;
        }
    }

    public boolean conectarBD() throws ExcepcionGD {
        try {
            this.conexion = DriverManager.getConnection(URL, USER, PASSWORD);
            return true;
        } catch (SQLException ex) {
            ExcepcionGD e = new ExcepcionGD();
            e.setCodigoErrorBD(ex.getErrorCode());
            e.setMensajeErrorBD(ex.getMessage());
            e.setMensajeErrorUsuario("Error general del sistema. Consulta con el administrador");
            throw e;
        }
    }
```

4. Se devuelve la información al usuario (Base de datos, Oracle)

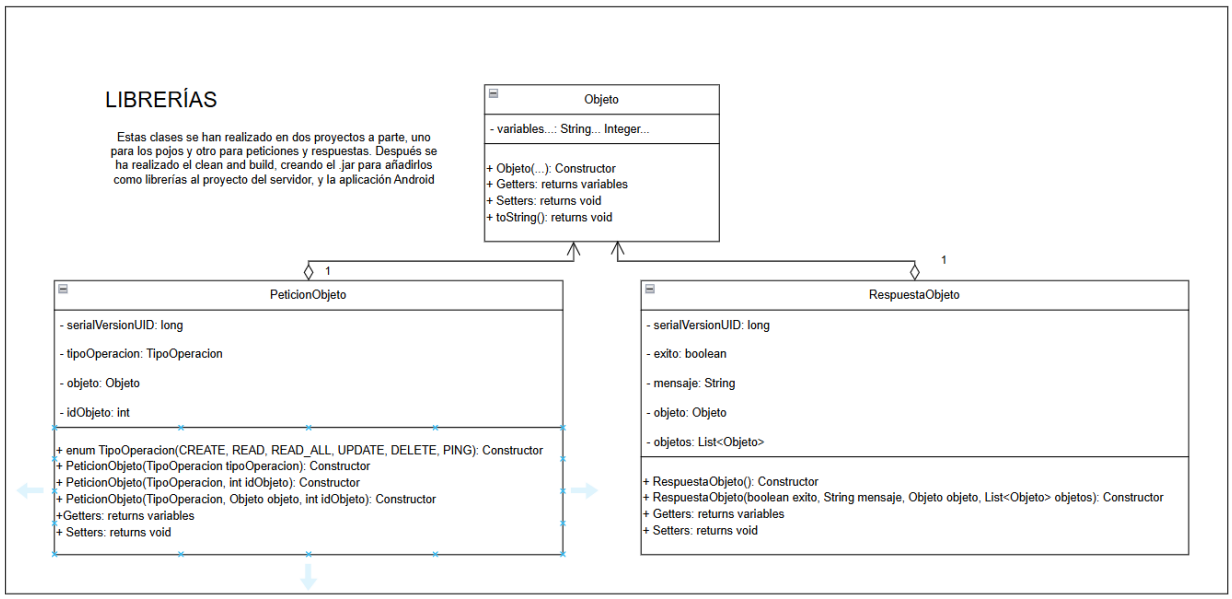


7.2 Diagrama de Clases

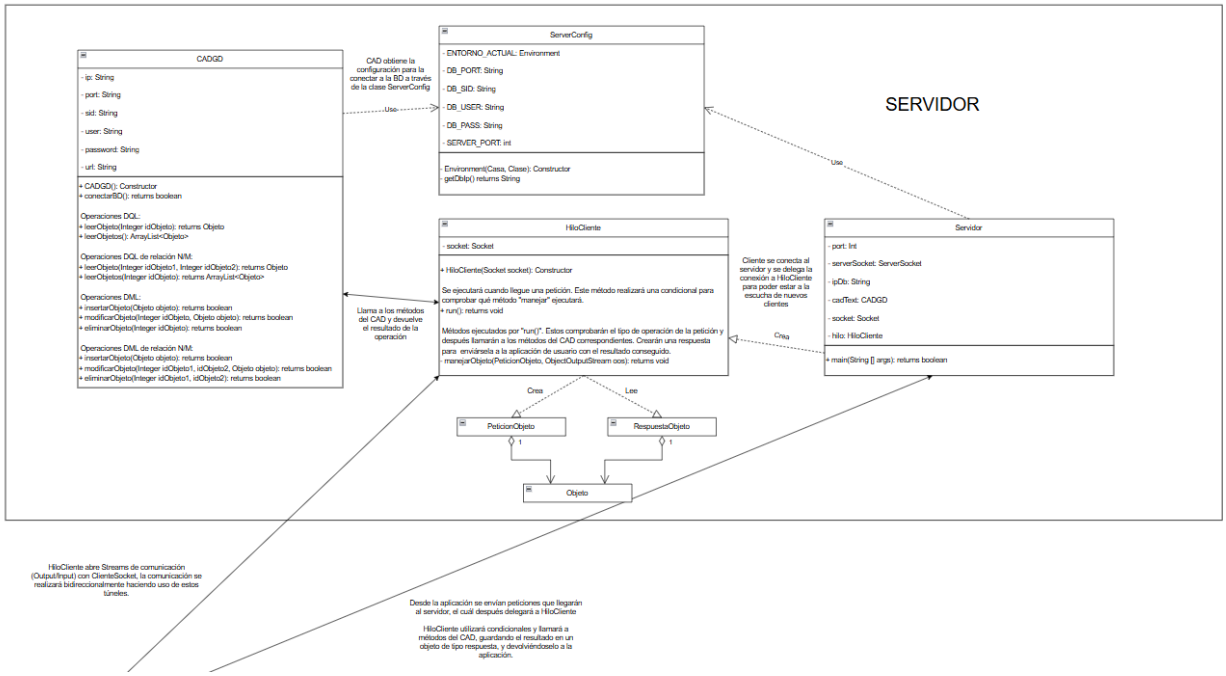
Debido al tamaño del diagrama, también se enviará en versión “.draw.io”. Este fichero se podrá abrir desde la página “draw.io”, yendo a "Archivo" > "Abrir desde" > "Dispositivo", y seleccionando el archivo. También se enviará en PNG.



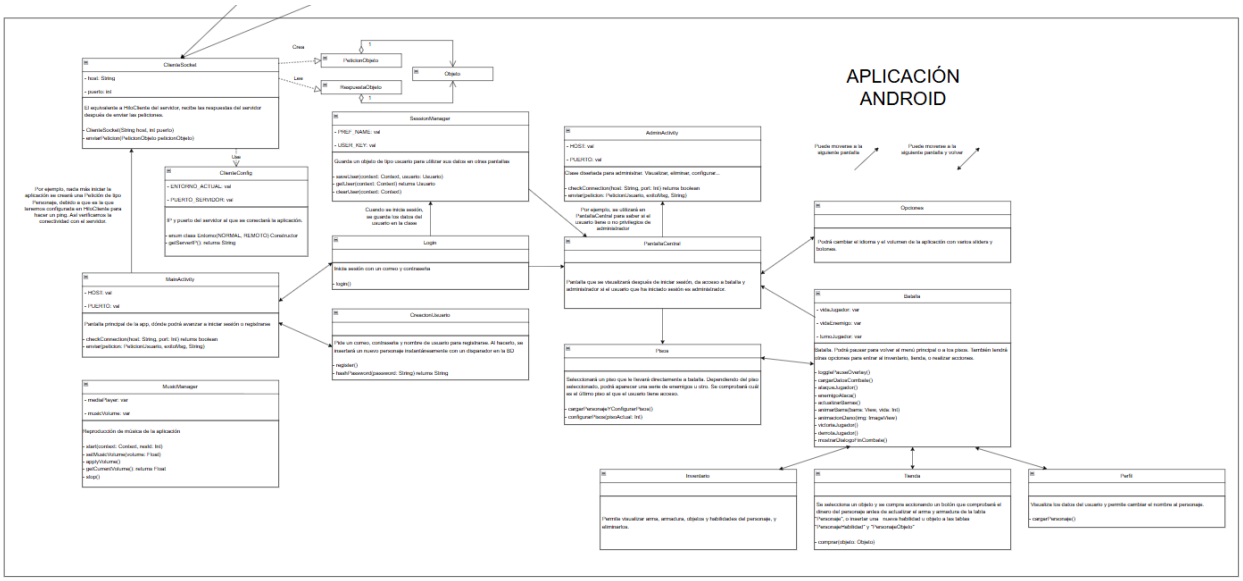
Librerías:



Servidor y conexión con la app:



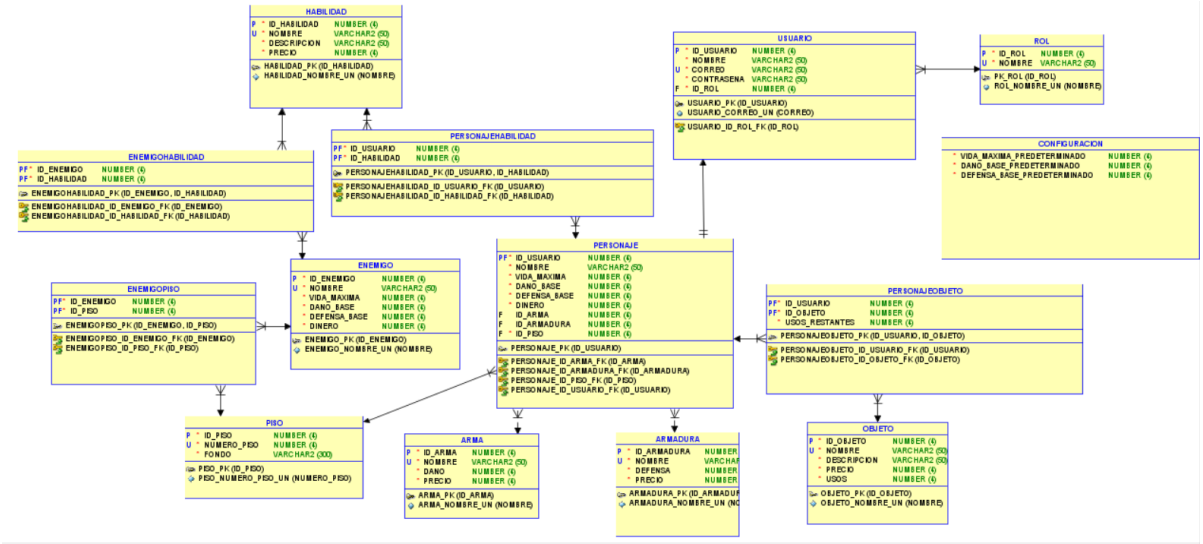
Aplicación Android:



PNG:

DiagramaDeClases_GodsBattle.drawio.png

7.3 Modelo Relacional



Todas las restricciones de negocio planteadas en el punto 6.4 se harán con “Check Constraints” (CC o CK “Check”). Excepto para la tabla “PersonajeObjeto” la cual tendrá CC y Trigger. También habrá diferentes procedimientos almacenados que servirán para actualizar datos de algunas tablas.

Tabla USUARIO

- usuario_ck_1: Verifica que el correo tenga un formato básico válido mediante la expresión LIKE '%_@_%._%'.
- MODIFICAR_USUARIO: Procedimiento que permite actualizar los datos de un usuario (nombre, correo, contraseña y rol) a partir de su identificador.

Tabla CONFIGURACION

- configuracion_ck_1: Garantiza que la vida máxima predeterminada sea mayor o igual a 0.
- configuracion_ck_2: Garantiza que el daño base predeterminado sea mayor o igual a 0.
- configuracion_ck_3: Garantiza que la defensa base predeterminada sea mayor o igual a 0.
- MODIFICAR_CONFIGURACION: Procedimiento que actualiza los valores predeterminados del sistema (vida máxima, daño base y defensa base).

Tabla PISO

- piso_ck_1: Garantiza que el número de piso sea mayor o igual a 0.

Tabla ARMA

- arma_ck_1: Garantiza que el daño del arma sea mayor o igual a 0.
- arma_ck_2: Garantiza que el precio del arma sea mayor o igual a 0.

Tabla ARMADURA

- armadura_ck_1: Garantiza que la defensa de la armadura sea mayor o igual a 0.
- armadura_ck_2: Garantiza que el precio de la armadura sea mayor o igual a 0.

Tabla OBJETO

- objeto_ck_1: Garantiza que el precio del objeto sea mayor o igual a 0.
- objeto_ck_2: Garantiza que el número de usos sea mayor o igual a 0.

Tabla HABILIDAD

- habilidad_ck_1: Garantiza que el precio de la habilidad sea mayor o igual a 0.

Tabla ENEMIGO

- enemigo_ck_1: Garantiza que la vida máxima del enemigo sea mayor o igual a 0.
- enemigo_ck_2: Garantiza que el daño base del enemigo sea mayor o igual a 0.
- enemigo_ck_3: Garantiza que la defensa base del enemigo sea mayor o igual a 0.
- enemigo_ck_4: Garantiza que el dinero del enemigo sea mayor o igual a 0.

Tabla PERSONAJE

- personaje_ck_1: Garantiza que la vida máxima del personaje sea mayor o igual a 0.
- personaje_ck_2: Garantiza que el daño base del personaje sea mayor o igual a 0.
- personaje_ck_3: Garantiza que la defensa base del personaje sea mayor o igual a 0.
- personaje_ck_4: Garantiza que el dinero del personaje sea mayor o igual a 0.
- MODIFICAR_PERSONAJE: Procedimiento que actualiza los atributos de un personaje, incluyendo estadísticas, dinero, y equipo y piso.

Tabla PERSONAJEOBJETO

- personajeobjeto_ck_1: Garantiza que los usos restantes del objeto sean mayor o igual a 0.
- MODIFICAR_PERSONAJEOBJETO: Procedimiento que permite actualizar la cantidad de usos restantes de un objeto asociado a un personaje.
- TRG_ELIMINAR_PERSONAJEOBJETO: Trigger que eliminará aquellos registros de la tabla cuya cantidad de usos sea 0.

8 MATRICES DE TRAZABILIDAD

Las matrices de trazabilidad constituyen una herramienta fundamental dentro del desarrollo de software, ya que permiten establecer una relación clara y directa entre los requisitos definidos en la fase de análisis y los distintos elementos que forman parte del sistema, como el diseño, la implementación o las pruebas. Su principal objetivo es asegurar que todos los requisitos han sido correctamente tenidos en cuenta durante el desarrollo del proyecto y que ninguno queda sin cubrir.

En el caso del proyecto GodsBattle, esta trazabilidad adquiere especial importancia debido a la variedad de funcionalidades que componen el sistema, como la gestión de usuarios, el sistema de combate por turnos, la progresión del personaje o la gestión de inventario. Todas estas funcionalidades han sido previamente definidas mediante requisitos funcionales, por lo que resulta necesario comprobar que cada uno de ellos tiene su correspondencia en el desarrollo del sistema.

	RF1	RF2	RF3	RF4	RF5	RF6	RF7	RF8	RF9	RF10	RF11	RF12
Iteración 0												
Iteración 1												
Iteración 2												
Iteración 3												
Iteración 4												

Por otro lado, este tipo de matrices resultan muy útiles a la hora de validar el sistema, ya que permiten comprobar de forma estructurada que cada requisito puede ser probado y verificado. De esta manera, no solo se garantiza que el sistema funciona correctamente, sino también que cumple con todos los objetivos definidos en la fase de análisis.

- Tipos de Tablas de Trazabilidad

En primer lugar: La trazabilidad entre requisitos y casos de uso permite comprobar que todas las funcionalidades definidas han sido representadas a nivel funcional, asegurando que el sistema contempla todos los comportamientos esperados.

En segundo lugar: La trazabilidad entre requisitos y diseño permite verificar que cada requisito tiene una correspondencia en la estructura del sistema, como puede ser una clase, módulo o componente, garantizando así que el diseño cubre todas las necesidades planteadas.

Por último: La trazabilidad entre requisitos y pruebas permite validar que cada requisito puede ser comprobado mediante uno o varios casos de prueba, lo que asegura que el sistema no solo está implementado, sino también correctamente verificado.

8.1 Requisitos: Casos de uso

Requisito	Descripción	Caso de uso
RF.1	Registro de usuario	CU01 - Registrar usuario
RF.2	Inicio de sesión	CU02 - Iniciar sesión
RF.3	Gestión de perfil	CU03 - Gestionar perfil
RF.5	Creación de personaje	CU04 - Crear personaje
RF.7	Inventario	CU05 - Gestionar inventario
RF.9	Progresión	CU06 - Gestionar progresión
RF.19	Iniciar combate	CU07 - Iniciar combate
RF.21	Cálculo de daño	CU08 - Resolver combate
RF.14	Objetos	CU09 - Usar objetos
RF.16	Armas	CU10 - Comprar equipamiento

8.2 Requisitos: Diseño (clases)

Requisito	Descripción	Elemento de diseño
RF.1	Registro de usuario	Clase Usuario, AuthController, BD Usuarios
RF.2	Inicio de sesión	AuthController, ClienteSocket, Servidor
RF.3	Gestión de perfil	Clase Usuario, Activity Perfil
RF.5	Creación de personaje	Clase Personaje, BD Personajes
RF.7	Inventario	Clase Inventario, Objeto
RF.8	Habilidades	Clase Habilidad
RF.9	Progresión	Clase Personaje, lógica de pisos
RF.10	Enemigos	Clase Enemigo
RF.13	Pisos	Clase Piso
RF.14	Objetos	Clase Objeto
RF.16	Armas	Clase Arma
RF.17	Armaduras	Clase Armadura
RF.19	Iniciar combate	CombateManager, Activity Combate
RF.21	Cálculo de daño	Clase Combate, lógica de cálculo
RF.22	Uso de habilidades	Clase Habilidad, lógica de combate
RF.23	Uso de objetos	Clase Objeto, Inventario
RF.28	Estadísticas	AdminController, BD equipamiento

8.3 Requisitos: Pruebas

Requisito	Caso de prueba
RF.1	CP01 - Registro correcto
RF.2	CP02 - Login válido
RF.19	CP03 - Inicio de combate
RF.21	CP04 - Cálculo de daño

En conclusión, las matrices de trazabilidad permiten garantizar la coherencia entre las distintas fases del proyecto, asegurando que todos los requisitos definidos han sido correctamente reflejados en el diseño, la implementación y las pruebas del sistema.

En el caso de GodsBattle, nuestra aplicación ha facilitado una visión estructurada del proyecto, permitiendo comprobar que todas las funcionalidades han sido tenidas en cuenta y correctamente desarrolladas. Además, estas matrices contribuyen a mejorar la calidad del sistema y a facilitar su mantenimiento, ya que permiten identificar de forma clara la relación entre los requisitos y los distintos componentes del sistema.

De esta manera, se asegura que el proyecto no solo funciona correctamente, sino que cumple con los objetivos planteados desde su fase inicial.